A sound pure type system with ranks as sorts and Type-in-Type

Pavel Sokolov[1] ‹ppsokolov@hse.ru›
Alexander Kuklev[2] ‹a@kuklev.com›

[1]HSE University, Laboratory of Theoretical Computer Science
[2]Radboud Univ. Nijmegen, Department of Software Science

16 October 2026

Abstract

We present a dependently typed pure type system which contains a circular axiom $s : s$ for a certain sort s and is nonetheless strongly normalising. Girard's paradox is avoided because our sorts are not closed under a Π-former: function types are placed in sorts according to their *rank*. Turns out that this system has a unique set of characteristics: it has similar normalisation behaviour as STLC (so complexity of normalisation is TOWER-complete) while at the same time allowing non-uniform representation of arbitrary elementary recursive functions; furthermore, it has a nontrivial set-theoretical model in ZF+GCH and allows to carry out nontrivial proof developments. Base normalisation proof is surprisingly simple (modulo general metatheoretical properties of pure type systems) and has been fully formalised in vanilla Rocq (formerly Coq) without any additional axioms.

1 Introduction

Pure type systems (PTSs, introduced independently by Berardi [17] and Terlouw [73]) are a family of minimalistic typed λ-calculi including grammar formers for variables $x \in \mathsf{Var}$, abstraction $\lambda(x : T).u$, application $u\,t$, types of dependent functions $\Pi(x : T).U$ (where U might depend on x) and *sorts* $s \in \mathcal{S}$ which act like "types of types".

Common syntax conventions to make it more lightweight are the following: application associates to the left, so $a\,b\,c$ is understood as $(a\,b)\,c$; in a sequence of abstractions, all λ symbols except the first can be omitted; $T \to U$ is an alias for $\Pi(x : T).U$ (where U does not depend on x); $(\to)$ is right-associative, so $T \to U \to V$ is understood as $T \to (U \to V)$. In addition, even though abstractions are always annotated with a type of an argument, we are going to omit it on paper whenever they are clear from the context. Also note that there

Variable	x	$\in$	X	
Sort	s	$\in$	S	
Term	T, U, t, u	$::=$	x	variable
		$\mid$	s	sort as type
		$\mid$	$\Pi(x\colon T).\,U$	type of dependent functions
		$\mid$	$\lambda(x\colon T).\,u$	anonymous function
		$\mid$	$u\,t$	function application

Figure 1: Grammar of pure type systems

is no syntactical distinction between terms and types — they are defined by a single grammar; an expression can act both as a term and as a type depending on a side where it stands next to the colon in the typing judgement $\Gamma \vdash _ : _$.

A single PTS is determined by two relations:

- *axiom* relation $\mathsf{Ax} \subseteq \mathcal{S} \times \mathcal{S}$ determines "to which sort(s) a given sort belongs as an element"; instead of $(s, s') \in \mathsf{Ax}$ it is customary to write simply $s : s'$.
- *rule* relation $\mathsf{Rule} \subseteq \mathcal{S}^2 \times \mathcal{S}$ places types of functions $\Pi(x : T).U$ into sorts accounting for the sorts where T and U are situated.

$$\textsc{Ax}\ \frac{}{\Gamma \vdash s : s'}(s, s') \in \mathsf{Ax} \qquad \textsc{Rule}\ \frac{\Gamma \vdash T : s_T \qquad \Gamma, x : T \vdash U : s_U}{\Gamma \vdash \Pi(x : T).\,U : s}(s_T, s_U, s) \in \mathsf{Rule}$$

$$\textsc{Var}\ \frac{}{\Gamma \vdash x : T}x : T \in \Gamma \qquad \textsc{Abs}\ \frac{\Gamma \vdash T : s_T \qquad \Gamma, x : T \vdash u : U}{\Gamma \vdash \lambda(x : T).u : \Pi(x : T).U}$$

$$\textsc{App}\ \frac{\Gamma \vdash u : \Pi(x : T).U \qquad \Gamma \vdash t : T}{\Gamma \vdash u\,t : U[t/x]} \qquad \textsc{Conv}\ \frac{\Gamma \vdash t : T \qquad \Gamma \vdash U : s \qquad T =_\beta U}{\Gamma \vdash t : U}$$

Figure 2: Typing rules of pure type systems

Systems from Barendregt's lambda cube [9, 10] fit naturally into the framework: for example, simply typed lambda calculus (STLC) [24] can be viewed as a pure type system with a single sort $*$, no axioms and a single rule $(*, *, *)$. In general, however, pure type systems are dependently typed since Π allows the codomain type to depend on the value of an argument — Coquand's Calculus of Constructions (CoC) [29] sits comfortably at the top of lambda cube and is modeled as a pure type system with two sorts $\{*, \Box\}$, a single axiom $* : \Box$ and four rules corresponding to simple functions $(*, *, *)$, polymorphic functions $(\Box, *, *)$, type operators $(\Box, \Box, \Box)$ and dependent types $(*, \Box, \Box)$.

As is known [71, 28, 32], both STLC and CoC are strongly normalising and

sound with respect to their set-theoretical semantics. A classical result shows that it is not true for arbitrary PTS.

Theorem (Girard'72 [33, 40])**.** *The following PTS called System U is not normalising:*

$$\mathcal{S} = \{*, \Box, \triangle\}, \quad \mathsf{Ax} = \{(*, \Box), (\Box, \triangle)\},$$
$$\mathsf{Rule} = \{(*,*,*), (\Box,*,*), (\triangle,*,*), (\Box,\Box,\Box), (\triangle,\Box,\Box)\}$$

Futhermore, the offending term has type $\Pi(P : *).P$*.*

This result is dubbed "Girard's paradox" since it renders System U inconsistent with respect to its intended logical semantics: inhabitedness of $\Pi(P : *).P$ would mean that every proposition is provable. Thus any system where System U embeds is inconsistent, too; in particular, an original version [50] of Martin-Löf's Intuitionistic Type Theory [52] embeds the following PTS known as "Type-in-Type" which is inconsistent due to Girard's paradox:

$$\mathcal{S} = \{*\}, \quad \mathsf{Ax} = \{* : *\}, \quad \mathsf{Rule} = \{(*,*,*)\}.$$

For a pure type system with usual set-theoretical semantics, it is natural to expect that a sort of *small types* $*$ is closed under Π, thus we cannot forbid the rule $(*,*,*)$; then, a circular axiom $* : *$ has to go. Long story short, it motivates an introduction of a *universe hierarchy* [51, 38] $\mathcal{S} = \{\mathcal{U}_k\}_{k\in\mathbb{N}}$ with axioms $\mathcal{U}_k : \mathcal{U}_{k+1}$ and rules $(\mathcal{U}_k, \mathcal{U}_l, \mathcal{U}_{k\vee l})$ for each k, l; when interpreted in sets, $[\![\mathcal{U}_k]\!]$ is taken to be the k-th inaccessible cardinal [77, 2].

Observing this, we took interest in the following two questions:

- If we were to drop the rule $(*,*,*)$ and keep the axiom $* : *$ instead, can we find a nontrivial consistent pure type system?
- Is there a consistent pure type system with total relations Ax and Rule whose natural set-theoretical semantics do not involve assumption of inaccessible cardinals?

Turns out that, for both questions, answer is "yes". Furthermore, both yes-s are witnessed by a single PTS, which we dub PTS-rk 🖒:

$$\mathcal{S} = \{\mathsf{rk}_r\}_{r\in\mathbb{N}} \quad \mathsf{Ax} = \{(\mathsf{rk}_r, \mathsf{rk}_0)\}_{r\in\mathbb{N}} \quad \mathsf{Rule} = \{(\mathsf{rk}_r, \mathsf{rk}_s, \mathsf{rk}_{(r+1)\vee s})\}_{r,s\in\mathbb{N}}$$

(Naming "rk" comes from *rank*, since rules compute precisely the rank of functions, which is the maximal nesting of Π in the domain type.)

Indeed, there is a circular axiom $\mathsf{rk}_0 : \mathsf{rk}_0$, and System U is not embeddable since rk_r is not closed under Π for any r. Also, as we will see in Section 3, our PTS-rk is strongly normalising despite an intuition that $\mathsf{rk}_r : \mathsf{rk}_0$ might "feel wrong". Modulo general metatheory of pure type systems laid out in Section 2, a normalisation proof turned out to be suspiciously simple, even more

so in comparison with normalisation proofs of other dependently typed systems [32, 12, 1]. To dispel our own doubts, we formalised a proof of normalisation in Rocq (formerly Coq) without using any additional axioms, yielding a fully constructive proof; our journey in formalisation is summarised in Section 3.1 and the whole development is available at `https://github.com/TurtlePU/pts-rk`.

Having completed the proof, we noticed that it exposes a primitive recursive procedure to normalise terms of PTS-rk; we dug further and discovered that, actually, there is an intimate connection between a class of elementary recursive functions [44, 60] and our system, as seen in Sections 4-5. As a byproduct of our study on complexity, we built a sound set-theoretic model for PTS-rk (see Section 6) which assumes *Generalised Continuum Hypothesis* [41, 39, 35, 36] which is an unusual pick for dependently typed systems where *inaccessible cardinals* are a more common assumption [77, 48]. Before concluding, we provide a sneak peek into what it is like to write proofs in PTS-rk and provide a full interpretation of Tarski–Mostowski–Robinson arithmetic R in PTS-rk (Section 7).

1.1 Use of artificial intelligence

At various points in our research, we interacted with GPT-6 Astra [54] to prove something for us. Sometimes it was a rehash of previously established results; sometimes it presented something new. In case we used the ideas hinted by LLM, we explicitly state so right next to the proof; even so, the text is completely human written since we wanted to make sure we completely understood the results (in addition, generated proofs always required additional explanations or a complete rewrite to make them understandable and simpler to read).

Also, we chatted with Claude Fable 5.1 [8] to find better points of reference.

1.2 Formalisation status

The metatheory developed below through strong normalisation for PTS-rk was formally verified in Rocq using handwritten proofs. These proofs are marked with a ; each icon is a hyperlink to the statement and proof in Rocq with version pinned at the moment of submission.

Additional formalised definitions and checked results, including developments generated with GPT-6 Astra, are marked with . These links also pin the source version. A badge on a definition links its formal encoding; on a result it links the checked proof. Where only a component of a paper claim is checked, the badge accompanies an explicitly delimited formalisation note, not the whole claim. In particular, the machine-level complexity classification and the derivation of all model premises from the stated set theory are not certified by these badges. External results cited in the introduction are not part of the verified metatheory.

2 Metatheory of pure type systems

For ease of presentation of our results in this and following sections, we assume a Barendregt convention [11] that all bound variables are distinct from free variables and from each other; furthermore, α-equivalence (renaming of bound variables) is assumed as a trivial equality on terms. Nevertheless, our formalisation takes variable treatment seriously; see Section 3.1 for more details.

2.1 PTS as a rewriting system

In Fig. 2, a rule of *conversion* (Conv) mentions *definitional equality* of types $T =_\beta U$. It is defined as a reflexive, symmetric, transitive closure of a one-step reduction.

Definition 1 (One-step reduction)**.** Relation of a one-step reduction $t \to_\beta t'$ is the smallest relation satisfying the following rules:

$$\frac{T \to_\beta T'}{\Pi(x : T).U \to_\beta \Pi(x : T').U} \qquad \frac{U \to_\beta U'}{\Pi(x : T).U \to_\beta \Pi(x : T).U'}$$

$$\frac{T \to_\beta T'}{\lambda(x : T).t \to_\beta \lambda(x : T').t} \qquad \frac{t \to_\beta t'}{\lambda(x : T).t \to_\beta \lambda(x : T).t'} \qquad \frac{t \to_\beta t'}{t\,u \to_\beta t'\,u}$$

$$\frac{u \to_\beta u'}{t\,u \to_\beta t\,u'} \qquad \text{Beta}\quad (\lambda(x : T).t)\,u \to_\beta t[u/x]$$

In essence, actual reduction happens only in rule Beta; other rules simply state the we can freely reduce any subterm.

The following result will be greatly useful in our development. It was proven many times over for different similar systems [26, 10]; we briefly revisit the proof since previous presentations usually did not allow reductions in the type of argument T in $\lambda(x : T).t$.

Theorem 1 (Characterisation of $(=_\beta)$)**.** *In arbitrary PTS,*

$$t =_\beta u \quad \Leftrightarrow \quad \exists v.\ t \to_\beta^* v \wedge u \to_\beta^* v,$$

where $(\to_\beta^*)$ *is reflexive transitive closure of* $(\to_\beta)$*.*

Proof. Note that it is enough to prove that $(\to_\beta^*)$ has a diamond property — for each t_0, t_1, t_2 such that $t_0 \to_\beta^* t_1$ and $t_0 \to_\beta^* t_2$ there exists a t' such that $t_1 \to_\beta^* t'$ and $t_2 \to_\beta^* t'$:

$$\begin{array}{ccc} t_0 & \longrightarrow & t_1 \\ \downarrow & & \vdots \\ t_2 & \dashrightarrow & t' \end{array}$$

Then, any "mountains" in the path $t =_\beta u$ can be switched into "valleys".

Now, diamond property is proven by standard technique of parallel reduction [50] which is defined as the smallest relation satisfying:

$$t \rightrightarrows t \qquad \frac{T \rightrightarrows T' \qquad U \rightrightarrows U'}{\Pi(x : T).U \rightrightarrows \Pi(x : T').U'} \qquad \frac{T \rightrightarrows T' \qquad t \rightrightarrows t'}{\lambda(x : T).t \rightrightarrows \lambda(x : T').t'}$$

$$\frac{t \rightrightarrows t' \qquad u \rightrightarrows u'}{t\,u \rightrightarrows t'\,u'} \qquad \frac{t \rightrightarrows t' \qquad u \rightrightarrows u'}{(\lambda(x : T).t)\,u \rightrightarrows t'[u'/x]}$$

Intuitively, while $(\to_\beta)$ is a single β-reduction somewhere in the term and $(\to_\beta)^*$ is a sequence of reductions, $(\rightrightarrows)$ is several simultaneous reductions immediately available in the term. Notice, then, that $(\to_\beta) \subseteq (\rightrightarrows) \subseteq (\to_\beta^*)$; by monotonicity of reflexive transitive closure, $(\to_\beta^*) \subseteq (\rightrightarrows^*) \subseteq (\to_\beta^*)^* = (\to_\beta^*)$, so $(\rightrightarrows^*) = (\to_\beta^*)$. Thus it is enough to prove diamond property of $(\rightrightarrows^*)$. Diamond property is preserved under reflexive transitive closure, so it is enough to prove diamond propery of $(\rightrightarrows)$, which holds by tedious case analysis. □

Setting aside practical applications of this theorem (to algorithmically check equivalence of terms, we can now search for common reduction result), it will prove useful already in our proof development. For example, the following "equality inversion" lemmas are direct corollaries:

Corollary 1 (Equality inversion 🫣)**.** *If $s =_\beta s'$, then $s = s' \in \mathcal{S}$; if $\Pi(x : T).U =_\beta \Pi(x : T').U'$, then $T =_\beta T'$ and $U =_\beta U'$.*

They will become necessary in proving subject reduction, or type preservation. But first, we will show some structural properties of a typing judgement given in Fig. 2.

2.2 Properties of a typing judgement

Again, they surely have been proven before for PTS [10, 32], but we slightly differ in the formulation of typing rules in important places so it is better to recheck that these hold.

Lemma 1 (Weakening 🦅)**.** *The following rule is admissible:*

$$\frac{\Gamma, \Delta \vdash t : T}{\Gamma, x : U, \Delta \vdash t : T}.$$

Proof. By structural induction on the typing derivation. □

Lemma 2 (Substitution 🦅)**.** *The following rule is admissible:*

$$\frac{\Gamma, x : U, \Delta \vdash t : T \quad \Gamma \vdash u : U}{\Gamma, \Delta[u/x] \vdash t[u/x] : T[u/x]}.$$

Proof. By structural induction on the typing derivation for t. For variables in Δ, it is necessary that both Δ and T get substitutions of u for x so Var is still applicable; for places in t where x used to be we need to apply weakening lemma to the typing derivation for u exactly $|\Delta|$ times. In the case of Conv it is important that definitional equality of types is preserved by substitution (thanks to characterisation of $(=_\beta)$). □

In addition, a family of so-called *inversion lemmas* [10, 16] stating that type of well-typed term is largely determined by shape of a term are also proven by straightforward induction on typing derivation (each one is proven separately, no mutual induction involved):

Lemma 3 (Inversion lemmas 🍣)**.**

$$\Gamma \vdash s : T \Rightarrow \exists s'.(s, s') \in \mathsf{Ax} \wedge T =_\beta s' \tag{1}$$

$$\Gamma \vdash \Pi(x : T).U : V \Rightarrow \exists s_T, s_U, s. \begin{cases} (s_T, s_U, s) \in \mathsf{Rule} \\ \Gamma \vdash T : s_T \\ \Gamma, x : T \vdash U : s_U \\ V =_\beta s \end{cases} \tag{2}$$

$$\Gamma, x : T, \Delta \vdash x : U \Rightarrow T =_\beta U \tag{3}$$

$$\Gamma \vdash \lambda(x : T).t : V \Rightarrow \exists s, U. \begin{cases} \Gamma \vdash T : s \\ \Gamma, x : T \vdash t : U \\ V =_\beta \Pi(x : T).U \end{cases} \tag{4}$$

$$\Gamma \vdash t\, u : V \Rightarrow \exists U, T. \begin{cases} \Gamma \vdash t : \Pi(x : U).T \\ \Gamma \vdash u : U \\ V =_\beta T[u/x] \end{cases} \tag{5}$$

To progress further, we need to make certain assumptions on the PTS we have in question.

Definition 2 (Functional PTS 🍣)**.** A pure type system is called *functional* if $\mathsf{Ax} \subseteq \mathcal{S} \times \mathcal{S}$ and $\mathsf{Rule} \subseteq \mathcal{S}^2 \times \mathcal{S}$ are functional relations, that is,

$$(s : s') \wedge (s : s'') \Rightarrow s' = s'' \quad \text{and} \quad (s_1, s_2, s) \in \mathsf{Rule} \wedge (s_1, s_2, s') \in \mathsf{Rule} \Rightarrow s = s'$$

Definition 3 (Total PTS 🍣)**.** A pure type system is called *total* if $\mathsf{Ax} \subseteq \mathcal{S} \times \mathcal{S}$ and $\mathsf{Rule} \subseteq \mathcal{S}^2 \times \mathcal{S}$ are total relations, that is,

$$\forall s. \exists s'.(s, s') \in \mathsf{Ax} \quad \text{and} \quad \forall s_1, s_2. \exists s.(s_1, s_2, s) \in \mathsf{Rule}$$

Theorem 2 (Uniqueness of typing 🍣)**.** *In a functional PTS, the following rule is admissible:*

$$\frac{\Gamma \vdash t : T \quad \Gamma \vdash t : T'}{T =_\beta T'}.$$

Proof. By structural induction on t and inversion of given typing derivations.

- $t = s \in \mathcal{S}$: by functionality of Ax.
- $t = \Pi(x : T).U$: inversion of typing gives a pair of (possibly different) sorts for T and another pair for U. By induction we get that ranks inside pairs are definitionally equal, which by equality inversion gives strictly equal sorts. Then what is needed holds by functionality of Rule.
- Case for $t = x \in \mathsf{Var}$ holds trivially.
- $t = (\lambda(x : T).u)$: by induction hypotheses and by congruence of $(=_\beta)$.
- $t = u\,v$: by typing inversion and by induction hypothesis for u we get $u : \Pi(x : T_1).U_1$ and $\Pi(x : T_1).U_1 =_\beta \Pi(x : T_2).U_2$. We need $U_1[v/x] =_\beta U_2[v/x]$ which we get from equality inversion and preservation of def.eq. under substitution.

□

In our proof of normalisation, we are going to rely on yet another property: if there is a derivation for $t : T$, then T is itself typeable with some sort. However, it does not hold in our current formulation of typing rules: in $\Gamma \vdash x : T$, nothing prevents a context Γ to contain an entry $(x : T')$ where T' is not typeable. Thus we need an additional assumption.

Definition 4 (Context well-formedness )**.** Context Γ is *well-formed* (written $\vdash \Gamma$) if every type in it is typed by some sort. Formally, set of well-formed contexts is the least set closed by the following rules:

$$\frac{}{\vdash \cdot} \quad \text{and} \quad \frac{\vdash \Gamma \quad \Gamma \vdash T : s}{\vdash \Gamma, x : T}$$

(where $\cdot$ denotes an empty context.)

Usually, when presenting a dependently typed system, context well-formedness is added as a premise to typing rules which would otherwise be premise-free (such as Ax and Var in a PTS). Here we instead provide it as an additional assumption only when necessary; in addition to leading to simpler metatheory (we avoid an otherwise mutually inductive definition of $(\vdash _)$ and $(_ \vdash _ : _)$), this is closer to how decision procedures for typing judgements are implemented.

Theorem 3 (Typing yields correct type )**.** *In a total PTS, if* $\Gamma \vdash t : T$ *and* $\vdash \Gamma$*, then there exists* $s \in \mathcal{S}$ *such that* $\Gamma \vdash T : s$*.*

Proof. By structural induction on typing derivation.

- Ax, Rule: by totality of Ax.
- Var: follows from $\vdash \Gamma$.
- Abs: by totality of Rule.

- App: $\Gamma \vdash t\,u : T[u/x]$. From induction hypothesis for t we know that $\Gamma \vdash t : \Pi(x : U).T : s$ for some s, then by typing inversion for $\Pi(x : U).T$ we get that T is typable with some sort. Then what is needed follows from substitution lemma.
- Conv: what is needed is present in the premise.

□

Totality is not a strictly necessary requirement — similar true propositions can be stated for non-total PTS, for example in CoC typing yields type which is either itself typable or equal to □ [10, 49]. But PTS-rk is indeed total so the theorem above is enough for our purposes.

Theorem 4 (Subject reduction 🔫)**.** *In a total PTS, the following rule is admissible:*

$$\frac{\Gamma \vdash t : T \quad \vdash \Gamma \quad t \to_\beta t'}{\Gamma \vdash t' : T}$$

Proof. By structural induction over derivation of $t \to_\beta t'$. Inversion of a typing judgement yields all necessary info, albeit because of it types are now given only up to definitional equality so we have to apply conversion rule in each case. Crucially, it requires the type we convert into to be typable by a sort, so that is where we use Theorem 3.

The only tricky case are reductions of a domain type in $\Pi(x : T).U$ and $\lambda(x : T).t$. Let us consider the case of the following reduction:

$$\lambda(x : T).t \to_\beta \lambda(x : T').t.$$

In order to complete this case, we have to prove that $\Gamma, x : T' \vdash t : U$ given that $\Gamma, x : T \vdash t : U$. By appropriately strengthening the statement, we get that it is enough to prove admissibility of the following rule called *context reduction*:

$$\frac{\Gamma, x : U, \Delta \vdash t : T \quad \vdash \Gamma, x : U, \Delta \quad U \to_\beta U'}{\Gamma, x : U', \Delta \vdash t : T}$$

The proof is, again, by induction on the typing derivation. The only non-trivial case is with Var rule used on x; it is taken care of by appropriate use of Conv rule to convert $x : U'$ to $x : U$. To use conversion, we have to prove that U is well-typed; thankfully, it follows from $\vdash \Gamma, x : U, \Delta$.

Note that context reduction is admissible in arbitrary PTS! We do not need it anywhere else, though, so we did not state it as a separate lemma. □

Characterisation of definitional equality, uniqueness of typing and subject reduction together imply the following corollary we are going to use a few times in our normalisation proof:

Corollary 2 (Type stability). *In a total functional PTS, the following holds:*

$$\begin{cases} \Gamma \vdash t : T \\ \Gamma \vdash t' : T' \\ \vdash \Gamma \\ t =_\beta t' \end{cases} \quad \Rightarrow \quad T =_\beta T'.$$

2.3 Properties of strongly normalising terms

We are almost done with classical results! What is left now is to state which terms are strongly normalising and collect properties of s.n. terms which allow our soundness proof to go through.

Informally, a term is strongly normalising iff there are no infinite reduction chains starting from it. In constructive setting and in formalisation, the following definition is a bit better [4, 59]:

Definition 5 (Strongly normalising term). Set of s.n. terms is the smallest set sn in which the following rule is valid:

$$\frac{\forall t', (t \to_\beta t') \Rightarrow t' \in \mathsf{sn}}{t \in \mathsf{sn}}.$$

In other words, a term is strongly normalising iff all possible reductions from it are strongly normalising.

Thus, terms which cannot be reduced further (also known as terms *in normal form*) are strongly normalising; then, terms which directly reduce into normal forms are also strongly normalising; and so on. Therefore, by this definition, strongly normalising terms are those that every reduction chain starting from them is finite.

The following properties are then provable by definition of sn and by structural induction on available derivations of sn for terms in question:

Lemma 4 (Structural properties of sn).

$$x, s \in \mathsf{sn} \qquad \frac{T \in \mathsf{sn} \qquad U \in \mathsf{sn}}{\Pi(x : T).U \in \mathsf{sn}} \qquad \frac{T \in \mathsf{sn} \qquad t \in \mathsf{sn}}{\lambda(x : T).t \in \mathsf{sn}} \qquad \frac{t\,u \in \mathsf{sn}}{t \in \mathsf{sn}} \qquad \frac{t[u/x] \in \mathsf{sn}}{t \in \mathsf{sn}}$$

Application, of course, is the only nontrivial case. In case of β-reduction, strong normalisation of both $(\lambda(x : T).t)$ and u does not imply anything about normalisation behaviour of $t[u/x]$: consider a classical example from untyped lambda calculus [11], $\omega = (\lambda(x : T).x\,x)$ and $\Omega = \omega\,\omega$; while obviously not typable, ω is strongly normalising, while Ω decidedly is not.

Situation changes when a head of application is not an abstraction at all. Thus we arrive at the definition of an *atomic term*.

Definition 6 (Atomic term). Set of atomic terms is the smallest set satisfying the following conditions:

1. s, $\Pi(x : S).T$ and y are atomic for all variables x,y, sort s and types S,T;
2. If t is atomic, then $t\,u$ is also atomic.

Atomic terms are useful precisely because their strong normalisation survives application:

Lemma 5. *If $t, u \in \mathsf{sn}$ and t is atomic, then $t\,u \in \mathsf{sn}$.*

Proof. By induction on derivations of sn for t and u. Consider possible redexes in $t\,u$: $t\,u$ is not itself a redex since t is atomic and hence not a lambda; the only possible redexes are either inside t or inside u. Applying corresponding induction hypotheses, we reach the desired result. The only caveat is that, for induction to go through, we need to check that atomicity of t is preserved by reduction; this is indeed true by induction on derivation of atomicity for t. □

A dual view on strong normalisation of application is the following: if we already know that $t[u/x] \in \mathsf{sn}$, then the most tricky case is already taken care of.

Lemma 6 (Weak head expansion [42, 58, 5]).

$$\begin{cases} t[u/x]u_1 \dots u_n \in \mathsf{sn} \\ U, u \in \mathsf{sn} \end{cases} \Rightarrow (\lambda(x : U).t)uu_1 \dots u_n \in \mathsf{sn}.$$

Proof. By induction on n.

- $n = 0$. We need to prove that if $t[u/x], u, U \in \mathsf{sn}$, then $(\lambda(x : U).t)u \in \mathsf{sn}$. Note that $t[u/x] \in \mathsf{sn}$ implies $t \in \mathsf{sn}$; then, by structural induction on derivations of sn for t, u, U we get what we want by considering all possible reductions.
- $n \to (n+1)$. By induction hypothesis, it is enough to show the following:

$$\begin{cases} (\lambda(x : U).t)uu_1 \dots u_n \in \mathsf{sn} \\ t[u/x]u_1 \dots u_{n+1} \in \mathsf{sn} \end{cases} \Rightarrow (\lambda(x : U).t)uu_1 \dots u_n u_{n+1} \in \mathsf{sn}.$$

$$This is true by structural induction on available evidence.$$

□

3 Strong normalisation of PTS-rk

Theorem 5 (Strong normalisation of PTS-rk).

$$\Gamma \vdash t : T \wedge \vdash \Gamma \Rightarrow t \in \mathsf{sn}.$$

Proof. By induction on the typing derivation. Case Conv is trivial; due to Lemma 4, cases Ax, Rule, Var and Abs present no problem as well. The only problematic case is application:

$$\begin{cases} \Gamma \vdash u : \Pi(x : T).U \\ \Gamma \vdash t : T \\ \vdash \Gamma \\ t, u \in \mathsf{sn} \end{cases} \Rightarrow u\,t \in \mathsf{sn}$$

Now, here is the part where stratification according to ranks comes in useful. As typing yields correct type, there is some r such that $\Gamma \vdash \Pi(x : T).U : \mathsf{rk}_r$. Let us explicitly add this to the list of premises; we will call the following proposition *Application Lemma*:

$$\mathrm{AL}_r \triangleq \quad \text{“} \begin{cases} \Gamma \vdash \Pi(x : T).U : \mathsf{rk}_r \\ \Gamma \vdash u : \Pi(x : T).U \\ \Gamma \vdash t : T \\ \vdash \Gamma \\ t, u \in \mathsf{sn} \end{cases} \Rightarrow u\,t \in \mathsf{sn}\text{”}$$

What indexing by r bought us is that now we can try proving this lemma by induction on r. Again, as it always is with strong normalisation of application, the problem is in case where application head is a lambda and a β-reduction takes place. This calls for another lemma, *Substitution Lemma*:

$$\mathrm{SL}_r \triangleq \quad \text{“} \begin{cases} \Gamma, x : T, \Delta \vdash u : U \\ \Gamma \vdash T : \mathsf{rk}_r \\ \Gamma \vdash t : T \\ \vdash \Gamma \\ t, u \in \mathsf{sn} \end{cases} \Rightarrow u[t/x] \in \mathsf{sn}\text{”}$$

A proof of AL_r for all r (which is what we are after) is done by complete mutual induction with SL_r. Thus there are two main parts:

- $(\forall s < r.\ \mathrm{SL}_s) \Rightarrow \mathrm{AL}_r$: by induction on derivations of $u, t \in \mathsf{sn}$. Consider possible reductions of $u\,t$: most of them happen either in u or in t, so that results of reductions from $u\,t$ are strongly normalising by inductive hypotheses. The only problematic case is when $u = \lambda(x : T').v$ and the reduction is β-reduction. By inversion of typing for u, it should be the case that $T' =_\beta T$; and by inversion of typing for $\Pi(x : T).U$, we get that $T : \mathsf{rk}_s$ and $U : \mathsf{rk}_t$ such that $r = \max(s + 1, t)$. Then, by type stability, $T' : \mathsf{rk}_s$, and furthermore, $s < r$, so we can apply substitution lemma.
- $(\forall s \leq r.\ \mathrm{AL}_s) \Rightarrow \mathrm{SL}_r$. So we need to prove $u[t/x] \in \mathsf{sn}$. We do it by simultaneous induction on derivation of $u \in \mathsf{sn}$ and structural induction on u. As substitution propagates through u, most cases are satisfied by Lemma 4. A funny case is when $u = x$: then, a premise that $t \in \mathsf{sn}$ helps.

The only problematic case is application $u = u_1\, u_2$; we consider three cases here:

- $u_1[t/x]$ is atomic — handled by Lemma 4.
- u_1 is not atomic — by virtue of it being well-typed, it means that $u = (\lambda(x : V).u')v_1 \dots v_k u_2$. All possible reductions from $u[t/x]$ are then handled by induction hypotheses.
- u_1 is atomic, but $u_1[t/x]$ is not — by virtue of u_1 being well-typed, the only case when this is possible is when $u_1 = xv_1 \dots v_k$, thus $u[t/x] = t(v_1[t/x]) \dots (v_k[t/x])(u_2[t/x])$. From induction hypotheses we know that all $v_i[t/x]$ and $u_2[t/x]$ are s.n.; t is s.n. by premise and its rank is r. Then $tv_1[t/x]$ is s.n. by application lemma for r; but note that rank of $tv_1[t/x]$ is not greater than r so application lemma applies again. Thus, by induction on k, the whole chain of applications is s.n.

□

This is a retelling of a proof that our friend, Aleksandr Levin, got after half-jokingly inputting the following prompt into GPT-6 Pro:

```
Consider the following pure type system:

S = { rk_n | n natural }
Axiom = { (rk_n, rk_0) | n natural }
Rule = { (rk_m, rk_n, rk_max(m+1,n)) | m, n natural }

Is it strongly normalizing or not?
```

After approximately 20 minutes, LLM outputted the proof which we retold above. Looking at it in retrospect, it seems as if we stratified functions by ranks with intent for such a direct proof to go through; honestly, we did not think of that and were eagerly trying to adapt well-known approaches for proving normalisation of pure type systems employing either saturated sets or candidates of reducibility [71, 33, 34, 32], which did not work because our "hierarchy" of ranks is not well-founded enough because of axioms $\mathsf{rk}_n : \mathsf{rk}_0$. This proof, however, does not even bother with it which means that we could pick *any* relation as Ax. Nevertheless, putting all sorts into rk_0 seems the most sensible approach to us since, informally, sort is kind of a constant which can not be converted into a function.

3.1 Formalising normalisation of PTS-rk in Rocq

In view of high technicality of the proof for strong normalisation which involves wrangling of unusual definition for strong normalisation and complex induction patterns (and in view of the fact that proof was supplied by the probabilistic machine), we decided to formalise it so that a sound simple algorithm would

check it for us. One of the natural choices for formalisation of programming languages is Rocq (formerly Coq) [74, 18], an interactive theorem prover based on Polymorphic, Cumulative Calculus of Inductive Constructions [68, 76]. As is the case with similar typed calculi, it is both a programming language and a higher-order logic, and correctness of a proof is checked by the type checker; in addition to that, Rocq features powerful metaprogramming facilities which allow writing custom *tactics* — programs written in a meta-language which generate terms of the target language, greatly simplifying the proofs. While set-theoretical semantics of Rocq assume an existence of countable hierarchy of inaccessible cardinals [77, 75], our formalisation never delves into the hierarchy and can be formalised in a much more restricted theory.

In order to formalise our strong normalisation proof, we first need to provide a general metatheory of PTS as laid out in Section 2. We could reuse existing projects on PTS formalisations such as [65, 3, 13, 30] but we wanted to explore the topic ourselves which led to a few minor differences discussed previously, thus we started from scratch.

When one formalises syntax with binders in a proof assistant, one of the most important decisions is how to represent binders. Here we went with making our terms indexed by the number of free variables it uses, a standard technique originating from [15, 19, 7]:

```
Inductive Term (L: Type) (n: nat) : Type :=
| sort: L -> Term L n
| Π: Term L n -> Term L (S n) -> Term L n
| var: Fin n -> Term L n
| lam: Term L n -> Term L (S n) -> Term L n
| app: Term L n -> Term L n -> Term L n.
```

(Inside our formalisation project, this definition is located here:)

Here, L is a type of sorts (L comes from *level* which is a name used for it in universe hierarchies), and n is a number of free variables. Entering new scope (such as in Π and in `lam`) adds one new free variable; variables, then, have to be numbers in range $[0; n)$, which are usually represented by a `Fin` type.

What is convenient in this representation is that it makes it harder to make mistakes when wrangling with context, defining substitution etc. — all of the techinical details which make working with syntax with binders so hard (authors of first implementation of such calculi, AUTOMATH [21, 22], remarked it being notoriously hard to get right). Also it has a certain category-theoretical feel to it: sufficiently generalised operations of renaming and substitution give the typal representation of syntax a structure of a *relative monad* [6, 7]. It is really hard to go wrong when implementing these operations (and the natural implementation turns variables into de Bruijn indices [21]); on top of them, usual substitution is easily definable.

Most of the proof development in Rocq mirrors what is written in Section 2; in

fact, we first wrote a proof in Rocq and then retold it in written English here, omitting tedious technicalities. The only considerable difference is in handling of the set of strongly normalising terms since, obviously, the proof of Substitution lemma given above does not follow the structure of inductive definition 5. Instead, we use a syntax-directed characterisation of the set of strongly normalising terms introduced in [43]:

Definition 7 (Syntax-directed characterisation of strong normalisation). Sets of terms SN (strongly normalising), SNx (strongly normalising neutral) and a relation $\to_{\mathsf{WHR}}$ (weak head reduction) are the least sets where the following rules are valid:

$$\frac{}{\mathsf{SNx}\, s}(s \in \mathcal{S}) \qquad \frac{\mathsf{SN}\, T \qquad \mathsf{SN}\, U}{\mathsf{SNx}(\Pi(x:T).U)} \qquad \frac{}{\mathsf{SNx}\, x} \qquad \frac{\mathsf{SNx}\, t \qquad \mathsf{SN}\, u}{\mathsf{SNx}\,(t\, u)}$$

$$\frac{\mathsf{SNx}\, t}{\mathsf{SN}\, t} \qquad \frac{\mathsf{SN}\, T \qquad \mathsf{SN}\, t}{\mathsf{SN}\,(\lambda(x:T).t)} \qquad \frac{t \to_{\mathsf{WHR}} u \qquad \mathsf{SN}\, u}{\mathsf{SN}\, t}$$

$$\frac{\mathsf{SN}\, U \qquad \mathsf{SN}\, u}{(\lambda(x:U).t)\, u \to_{\mathsf{WHR}} t[u/x]} \qquad \frac{t \to_{\mathsf{WHR}} t'}{t\, u \to_{\mathsf{WHR}} t'\, u}$$

In a sense, SNx is a description of atomic strongly normalising terms; SN adds a case for lambdas and for weak head expansion, covering all possible cases. Existence of such sets is by virtue of them being subsets of the set of terms, so such mutual inductive definition is not problematic.

One can prove that set SN is equal to sn; we, however, need only the following:

Lemma 7 (Soundness of SN). $\mathsf{SN} \subseteq \mathsf{sn}$.

Proof. By mutual induction on the evidence $\mathsf{SN}\, t$ for arbitrary t. To do this, we also have to provide a term-indexed proposition we are proving for SNx and a relation on terms we are proving for WHR:

- $\forall t, \mathsf{SNx}\, t \Rightarrow \mathsf{sn}\, t$.
- $\forall t\, t', (t \to_{\mathsf{WHR}} t') \Rightarrow (t \to_{\mathsf{WHR}-\mathsf{sn}} t')$ (where the last relation is defined in the same way as WHR, but the head contains evidence of sn instead of SN.)

Proof is extremely simple: by Lemmas 4,5,6. We just have to notice that if term is SNx, it is atomic, so lemma 5 applies in the relevant case. □

Then, to prove strong normalisation, it is enough to prove that if term is typable, it is SN; we only have to state Application Lemma and Substitution Lemma in terms of SN, and the proof goes through.

Lemma 8. AL_s *for all* $s \leq r$ *implies* SL_r.

Proof. We have to prove that $\mathsf{SN}\, t$ and $\mathsf{SN}\, u$ imply $\mathsf{SN}\,(t[u/x])$. We do so by mutual induction on evidence for t with the following propositions proven for SNx and WHR:

- $\mathsf{SNx}\, t \land \Gamma \vdash t : T \Rightarrow \mathsf{SNx}\,(t[u/x]) \lor (\mathsf{SN}\,(t[u/x]) \land \exists s \leq r.\ \Gamma \vdash T : \mathsf{rk}_s)$;
- $(t \to_{\mathsf{WHR}} t') \Rightarrow (t[u/x] \to_{\mathsf{WHR}} t'[u/x]) \land (\mathsf{SN}\,(t'[u/x]) \Rightarrow \mathsf{SN}\,(t[u/x]))$.

The proof then follows as written in a non-formal treatment above. □

4 Computational complexity of PTS-rk

In this section we prove that convertibility of closed well-typed PTS-rk terms is TOWER-complete.

The Rocq development checks fixed-rank normalisation, an automatic rank bound and a tower bound on the normaliser's cost, as well as a Church-style STLC reduction. Costs count the explicitly defined operations on annotated syntax, with unary sort and variable indices; no machine-level complexity-class formalisation is claimed. The representability results in the following section are not formalised.

As requested further, LLM confessed that it was largely inspired by studies on complexity of normalisation for STLC [70, 62, 14, 53]. And indeed, upon taking a closer look at the proof of Theorem 5, one cannot help but notice it actually being a primitive recursive algorithm [66, 46, 56] for term normalisation in disguise. Or something a tad faster:

Definition 8 (d-bounded terms 🤔)**.** A PTS-rk term is d-bounded if it is well-typed in some well-formed context and heads of all redexes in it have rank equal to or below d.

Theorem 6 (Elementary recursive normalisation of bounded PTS-rk 🤔)**.** *For each d, there is an elementary recursive algorithm [60] for finding a normal form of d-bounded terms.*

The linked theorem assumes the original typing judgement, context well-formedness and the redex-rank bound separately. It gives a normal form of the same type and cost at most $\mathrm{tower}(2d+1, \mathrm{nodes}(t) + 5d + 2)$ in the operation-count model just described.

Proof. Consider the following relation of "greedy parallel reduction" (implemented as the function `greedy`) 🤔:

$$x \Rightarrow x \qquad s \Rightarrow s \qquad \frac{T \Rightarrow T' \qquad U \Rightarrow U'}{\Pi(x:T).U \Rightarrow \Pi(x:T').U'} \qquad \frac{T \Rightarrow T' \qquad t \Rightarrow t'}{\lambda(x:T).t \Rightarrow \lambda(x:T').t'}$$

$$\frac{t \Rightarrow (\lambda(x:T).t') \qquad u \Rightarrow u'}{t\, u \Rightarrow t'[u'/x]} \qquad \frac{t \Rightarrow t' \qquad t' \neq \lambda\ldots \qquad u \Rightarrow u'}{t\, u \Rightarrow t'\, u'}$$

It differs from usual parallel reduction since it greedily applies every "many-argument" redex available in a term. For example, $(\lambda(x : X).\lambda(y : Y).\lambda(z : Z).t)\, a\, b\, c$ gets reduced to $t'[a/x][b/y][c/z]$ in a single step. This relation has the following properties:

- $(\Rightarrow) \subseteq (\to_\beta^*)$, so it is several usual reductions baked into one.
- This relation is total and functional.
- This relation can be realised with an elementary recursive algorithm $\mathcal{A}$: traverse the term bottom-up, immediately applying every β-redex which is formed when result of a recursive run of algorithm on application head is a lambda.
- If $t \Rightarrow t'$, then $|t'| \leq (2+\varepsilon)^{\mathcal{O}(|t|)}$.
- If redex heads in t have ranks less than *or equal to* d and $t \Rightarrow t'$, redex heads in t' have ranks *less than* d: new redexes appear only after substitution of abstractions into application heads, and ranks of these abstractions are strictly less than the rank of redex heads in which these substitutions happened.

Because of the last property, it is enough to run $\mathcal{A}$ for d times in total to normalise a d-bounded term; since input size on k-th run is bounded by k-th tower of exponents, the whole algorithm is elementary recursive.

To make sure that the corresponding function is total, output 0 if result of an algorithm contains a β-redex. □

Similar result was already proven for STLC [62, 14] since simple types can be classified by ranks as well. And since rank is easily computable for simple types, normalisation of arbitrary STLC terms is in TOWER. A little weaker result holds for our dependently typed PTS:

Theorem 7. *Convertibility of* ***closed*** *well-typed* PTS-rk *terms is in* TOWER.

Proof. By Theorem 6, it is enough to compute some upper bound B on the rank of types appearing in abstractions of a term, then run a normaliser for $(B+1)$-bounded terms and compare the results; B has to be bound by an elementary recursive function on the size of input. Consider, then, the following elementary recursive functions to compute such upper bound, written in pseudocode and using a suitable encoding of terms, tuples and lists. The estimator itself is translated to Rocq; this definition badge does not assert its semantic bound (see the formalisation note below):

```
maxRankLiteral rkᵣ      = r
maxRankLiteral Π(x:T).U =
    maxRankLiteral T ⊔ maxRankLiteral U
maxRankLiteral x        = 0
maxRankLiteral λ(x:T).t =
```

```
    maxRankLiteral T ⊔ maxRankLiteral t
maxRankLiteral (t u)    =
    maxRankLiteral t ⊔ maxRankLiteral u

traversal _ rk_r       _ = (r , 0 , 0)
traversal Γ Π(x:T).U d =
    let (v^T , r^T , a^T) = traversal Γ T d
        (v^U , r^U , a^U) = traversal (x := v^T, Γ) U d
     in (v^U , suc r^T ⊔ r^U , a^T ⊔ a^U)
traversal Γ x         d = (d , Γ[x] , 0)
traversal Γ λ(x:T).t d =
    let (v^T , r^T , a^T) = traversal Γ T d
        (v^t , r^t , a^t) = traversal (x := v^T, Γ) t d
     in (v^t , r^t , r^T ⊔ a^T ⊔ a^t)
traversal Γ (t u)    d =
    let (v^t , r^t , a^t) = traversal Γ t d
        (_   , _   , a^u) = traversal Γ u d
     in (v^t , r^t , a^t ⊔ a^u)

estimateRank t =
    let (v^t , r^t , a^t) =
        traversal [] t (maxRankLiteral t)
     in a^t
```

Function `maxRankLiteral` t computes maximal rank which is contained as a subterm in term t. The workhorse of this algorithm, `traversal` Γ t d, accepts the list Γ of ranks in context of a term t and a "default rank" d, and outputs the triple (v, r, a), where v is value of t as a rank (if supplied with enough arguments), r is its rank if t is a type (if supplied with enough arguments) and a is an upper bound on rank of types appearing in abstractions which we are trying to compute. The only suspicious case is `traversal` $\Gamma\, x\, d$ where we output d (the "default" rank) as a value of x: could we compute value of x instead, if present? Regrettably, this would require normalisation of a term corresponding to x, which is too long; instead we notice that rk_r can be reduced only from terms which already contain it as a subterm, so value of x as a rank can only be rk_r which is already present somewhere in original term. And our result is monotone with respect to the value we output here so it is enough to provide some upper bound on it; thus it is enough to output a maximal rank which occurs in original term as a subterm. □

Checked normalisation bound. The Rocq normaliser has a checked tower upper bound. Its rank bound is computed in linear operation count. This proof uses the conservative bound $1 + \texttt{maxRankLiteral}(t) + \mathrm{nodes}(t)$, not the displayed `estimateRank` argument: the development includes a typed counterexample to using $\texttt{estimateRank}(t)+1$ as a redex-head rank bound. These badges certify the replacement normalisation bound, not the estimator argu-

ment above or a machine-level convertibility theorem.

Note that closedness condition is unavoidable: without it, we would not know ranks of type variables from context and there would be no known bound to the rank of redexes.

Theorem 8. *Convertibility of closed well-typed* PTS-rk *terms is TOWER-hard (hence TOWER-complete).* [1]

Proof. By reduction from convertibility of Curry-style STLC terms, since that problem is TOWER-hard [53].

Just in case, we start from Curry-style STLC terms. We then reduce this task to convertibility of **closed** Curry-style STLC terms by adding λ-abstractions to close over free variables; finally, we infer types of variables in an STLC term by a linear unification algorithm referenced in [53] and settle on **closed** well-typed **Church-style** STLC terms. The following translation then yields **open** well-typed PTS-rk terms:

$$\begin{aligned} (\!|x|\!) &= x \\ (\!|\lambda x^{\sigma}.t|\!) &= \lambda(x : \sigma).(\!|t|\!) \\ (\!|t\ u|\!) &= (\!|t|\!)\ (\!|u|\!) \end{aligned}$$

The problem is that types σ might contain type variables $T_1, \ldots, T_n$ which are not bound. However, it is easily fixable by introducing missing types via abstractions:

$$t \rightsquigarrow \lambda(T_1 : \mathsf{rk}_0) \ldots (T_n : \mathsf{rk}_0).(\!|t|\!)$$

Assigning all type variables rank rk_0 preserves well-typedness since values of such types were never treated as functions in original term. □

Corollary 3. *Type checking for closed* PTS-rk *terms is TOWER-complete.*

Proof. The following term demonstrates that convertibility of closed well-typed terms $t, t' : T$ reduces to type checking:

$$\lambda(P : T \to \mathsf{rk}_0)(x : P\,t).(\lambda(y : P\,t').y)\,x$$

This term is well-typed iff $P\,t =_\beta P'_t$ which holds iff $t =_\beta t'$.

[1] The badge certifies the annotation-preserving reduction from closed Church-style STLC, including typing, preservation and reflection of conversion, and the exact additive encoding-size bound. The source TOWER-hardness theorem and the Curry-to-Church preprocessing below are external to this formalisation; the badge does not certify TOWER-completeness itself.

Then, we only have to prove that type checking for PTS-rk is in TOWER. Take a look at the following syntax-directed type synthesis rules:

$$\textsc{Ax'}\ \frac{}{\Gamma \vdash \mathsf{rk}_r \Rightarrow \mathsf{rk}_0} \qquad \textsc{Rule'}\ \frac{\Gamma \vdash T \Rightarrow \mathsf{rk}_r \qquad \Gamma, x : T \vdash U \Rightarrow \mathsf{rk}_s}{\Gamma \vdash \Pi(x : T).\, U \Rightarrow \mathsf{rk}_{\max(r+1,s)}}$$

$$\textsc{Var'}\ \frac{}{\Gamma \vdash x \Rightarrow T} x : T \in \Gamma \qquad \textsc{Abs'}\ \frac{\Gamma \vdash T \Rightarrow \mathsf{rk}_r \qquad \Gamma, x : T \vdash u \Rightarrow U}{\Gamma \vdash \lambda(x : T).u \Rightarrow \Pi(x : T).U}$$

$$\textsc{App'}\ \frac{\Gamma \vdash u \Rightarrow \Pi(x : T).U \qquad \Gamma \vdash t \Rightarrow T' \qquad T =_\beta T'}{\Gamma \vdash u\, t \Rightarrow U[t/x]}$$

This relation is functional (in the sense that if $\Gamma \vdash t : T$ and $\Gamma \vdash t : T'$, then $T = T'$ as words) and the corresponding partial function is realised by an algorithm in TOWER: this algorithm recurses on the term t, matches on the inferred types as needed and collects equalities on types it has to check. Then, it can naïvely check all equalities in bulk: there is at most $\mathcal{O}(|t|)$ such equalities on terms of size $\mathcal{O}(2^{\mathrm{poly}(|t|)})$, so this runs in TOWER by Theorem 7. To finish the proof, it is enough to notice that

$$\Gamma \vdash t : T \quad \iff \quad \exists T', r.\ (\Gamma \vdash T : r) \wedge (T =_\beta T') \wedge (\Gamma \vdash t \Rightarrow T').$$

So the main algorithm infers the type of T, checks that it is rk_r for some r, infers the type of t and compares it with T. □

5 PTS-rk as a computational model

In this section we prove that functions representable in PTS-rk are, under minor assumptions, precisely elementary recursive functions, and provide a bunch of other, more specialised, results.

Another consequence of Theorem 6 is a natural upper bound for functions *uniformly* representable in PTS-rk (this result is also similar to the one for STLC [63, 70, 31]).

Definition 9. A family of functions on natural numbers $\mathcal{F}$ is *uniformly* representable in PTS-rk if there is an *input natural number type* $N_I : \mathsf{rk}_r$ and an *output natural number type* $N_O : \mathsf{rk}_s$ together with *encodings* $\forall n \in \mathbb{N}.[n]^I : N_I$ and $\forall n.[n]^O : N_O$ such that all of them are in normal form and for each $(f : \mathbb{N}^k \to \mathbb{N}) \in \mathcal{F}$ there is a *representation* $F : N_I \to \ldots \to N_I \to N_O$ which is also in normal form and such that

$$\forall (n_1, \ldots, n_k) \in \mathbb{N}^k : \quad F\,[n_1]^I \ \ldots\ [n_k]^I =_\beta [f(n_1, \ldots, n_k)]^O$$

Remark. Note that requirement for terms to be in normal form is not restrictive: we can always normalise a non-normal term.

Theorem 9. *A family of elementary recursive functions is* ***not*** *uniformly representable in* PTS-rk.

Proof. By direct diagonalisation argument. Suppose we could encode all elementary recursive functions with $N_I : \mathsf{rk}_r$ and $N_O : \mathsf{rk}_s$.

By Theorem 6, normalisation of $(r+1)$-bounded terms $\mathcal{SN}_{r+1}$ is elementary recursive, so the following function is elementary recursive as well:

$$h(t) = \begin{cases} 1, & \mathcal{SN}_{r+1}(t\,[\mathtt{t}]^I) = [0]^O, \\ 0 & \text{otherwise.} \end{cases}$$

(Where $\mathtt{t} \in \mathbb{N}$ is code of a term t used in the definition of h.)

By assumption, it is encodable by $H : N_I \to N_O$. Observe the value of $h(H)$:

$$h(H) = 1 \Leftrightarrow \mathcal{SN}_{r+1}(H\,[\mathtt{H}]^I) = [0]^O \Leftrightarrow H\,[\mathtt{H}]^I =_\beta [0]^O \Leftrightarrow h(H) = 0$$

First equivalence is by definition of h; second is by virtue of expression under $\mathcal{SN}$ being $(r+1)$-bounded; the last is by definition of h being encoded by H. $\square$

A logical next question is then: which functions are computable? Elementary recursive! Though, of course, not in the same sense as before.

Definition 10 (Polymorphic numeral family)**.** For each fixed $j \geq 0$, there is a type of *polymorphic Church numerals* of rank $j+2$:

$$\mathcal{N}_j \triangleq \Pi(X : \mathsf{rk}_j).(X \to X) \to X \to X;$$

and for each fixed n, there is a corresponding *Church literal* of type $\mathcal{N}_j$:

$$\overline{n} \triangleq \lambda\, X\, f\, x.f^n\, x.$$

($f^n\, x$ means repeated application of f to x: $f(\ldots(f\, x)\ldots)$.)

Theorem 10 (Non-uniform elementary representability)**.** *For each elementary recursive function $f : \mathbb{N}^n \to \mathbb{N}$, there is a* cost *$c_f$ such that for each* base rank *j, there is a* representation *$F_j : \mathcal{N}_{j+c_f} \to \ldots \to \mathcal{N}_{j+c_f} \to \mathcal{N}_j$ such that*

$$\forall (x_1, \ldots, x_n) \in \mathbb{N}^n : \quad F_j\, \overline{x_1}\, \ldots\, \overline{x_n} =_\beta \overline{f(x_1, \ldots, x_n)}.$$

Proof. As is well-known, simply typed Church numerals admit addition and multiplication [25, 11]. It is still true for our rank-refined version:

$$\begin{array}{lcll} \mathsf{Add}_j & \triangleq & \lambda\, m\, n\, X\, f\, x.m\, X\, f\, (n\, X\, f\, x) & : \mathcal{N}_j \to \mathcal{N}_j \to \mathcal{N}_j \\ \mathsf{Mul}_j & \triangleq & \lambda\, m\, n\, X\, f\, x.m\, X\, (n\, X\, f)\, x & : \mathcal{N}_j \to \mathcal{N}_j \to \mathcal{N}_j \end{array}$$

By substitution-basis theorem [57], it is enough to additionally provide modulus $x \bmod y$ (where $x \bmod 0 = x$) and binary exponentiation 2^x since substitution, constants and projections are "free" in our calculus.

By Theorem 9, we cannot hope to have both remaining operations represented uniformly. Indeed, naïve import of traditional exponentiation term for untyped Church numerals [25, 61] raises the rank of an exponent:

$$\mathsf{Exp}_j \triangleq \lambda\, m\, n\, (X : \mathsf{rk}_j)\, f\, x.\, n\, (X \to X)\, (\lambda (h : X \to X)(y : X).m\, X\, h\, y)\, f\, x \\ : \mathcal{N}_j \to \mathcal{N}_{j+1} \to \mathcal{N}_j$$

And that is exactly the reason we allowed input and output ranks of Church numerals to differ.

There still remains a problem, however: by allowing ranks to differ, we lost uniformity of substitution. In particular, consider an elementary function $x \mapsto x + 2^x$. To represent such computation using combinators we defined, x has to have both type $\mathcal{N}_j$ (as left addend) and $\mathcal{N}_{j+1}$ (as exponentiation power). Instead of introducing subtyping in the system, we make use of the following family of *numeral-preserving lowerings*:

$$\delta_j \triangleq \lambda\, n\, X\, f\, x.\, n\, (X \to X)\, (\lambda\, h\, y.f\, (h\, y))\, (\lambda\, y.y)\, x : \mathcal{N}_{j+1} \to \mathcal{N}_j$$

As the name implies, the goal of such combinators is to lower the rank of a numeral, but preserve its "value". It is achieved in the following way: given an f and x for lower-ranked type, an original combinator for n is used to build an n-fold composition of f with itself which is then applied to x.

Thus we can lower ranks arbitrarily: $\delta_{b,a} \triangleq \delta_a \circ \ldots \circ \delta_{b-1} : \mathcal{N}_b \to \mathcal{N}_a$. This restores substitution and allows gluing representation of elementary recursive functions together; cost c_f of a function can be computed from any expression that represents it — addition and multiplication cost 0, exponentiation costs 1, operands are merged by taking the maximum of their costs. (Of course, cost is not unique and depends on the way function is represented with an expression.)

All that is left, then, is to represent modulus. General idea is as follows: first, suppose that we are dealing with a simpler problem of taking modulus by a constant modulo, e.g. $\bmod 5$. Notice that there are only 5 possible answers: 0,1,2,3,4. As such, they can be represented by the type which contains exactly 5 elements; Church encoding [20] yields a type of *selectors*:

$$V_5 \triangleq \Pi(X : \mathsf{rk}_j).\, X \to X \to X \to X \to X \to X : \mathsf{rk}_{j+1}$$

Its 5 different canonical values simply pick one of 5 inputs, to each their own.

There is an operation of *cyclic shift* on selectors:

$$c_5 \triangleq \lambda\, s\, X\, x_1\, x_2\, x_3\, x_4\, x_5.\, s\, X\, x_2\, x_3\, x_4\, x_5\, x_1 : V_5 \to V_5$$

Iterating it n times on some pinned "zeroth" selector, we actually get the $(n \bmod 5)$-th selector. All that is left is to read the numeral back:

$$\rho \triangleq \lambda\, s\, X\, f\, x.\, s\, X\, (f^0\, x)\, (f^1\, x)\, (f^2\, x)\, (f^3\, x)\, (f^4\, x) : V_5 \to \mathcal{N}_j$$

Now, in order to model $m \bmod n$ in this vein, we need to *generate* V_n dynamically. Lucky for us, PTS-rk is dependently typed so there is hope.

And indeed, the following construction was produced in discussion with GPT-6 Astra.

We are going to produce a combinator $\mathsf{Mod} : \mathcal{N}_{j+1} \to \mathcal{N}_{j+4} \to \mathcal{N}_j$. Fix parameter $X : \mathsf{rk}_j$ and consider the following signature for abstract $S : \mathsf{rk}_{j+1}$:

$$\begin{array}{ll} K : X \to S, & z : S, \\ L : (X \to S) \to X \to S, & c : S \to S, \\ \rho : S \to (X \to X) \to X \to X, & r : (X \to X) \to (X \to X). \end{array}$$

Intention is that S is simply-typed V_q for X, z is its "zero", K is a "zero" for V_{q+1}, c is a cyclic shift for V_q, L is a cyclic shift for V_{q+1}, ρ converts a selector into a numeral, and r is the result of modulus computed using structure from the previous step (this is similar to the trick used to define a predecessor function).

A callback accepting such structure and returning X has the following type:

$$\begin{aligned} H_X \triangleq \Pi(S : \mathsf{rk}_{j+1}).\ (X \to S) \to S \to ((X \to S) \to X \to S) \to (S \to S) \to \\ (S \to (X \to X) \to X \to X) \to ((X \to X) \to X \to X) \to X \quad : \mathsf{rk}_{j+3} \end{aligned}$$

A type of "records" is thus $P_X \triangleq H_X \to X : \mathsf{rk}_{j+4}$, with a constructor

$$\langle S, K, z, L, c, \rho, r \rangle \triangleq \lambda h.\, h\, S\, K\, z\, L\, c\, \rho\, r.$$

As an initial P_X, we build a structure for V_1 and an r which behaves like $m \bmod 0 = m$:

$$\begin{array}{ll} S_0 \triangleq X \to X, & \\ K_0 \triangleq \lambda(a : X)(b : X).a, & z_0 \triangleq \lambda(a : X).a, \\ L_0 \triangleq \lambda(t : X \to S_0)(a : X)(b : X).t\, b\, a, & c_0 \triangleq \lambda(s : S_0).s, \\ \rho_0 \triangleq \lambda(s : S_0)(g : X \to X)(a : X).s\, a, & r_0 \triangleq \delta_j\, m\, X. \end{array}$$

Pack them into $p_0 \triangleq \langle \ldots \rangle$.

Then, given a structure for q, a structure for $q + 1$ is:

$$\begin{array}{ll} S^+ \triangleq X \to S, & \\ K^+ \triangleq \lambda(a : X)(b : X).Ka, & z^+ \triangleq K, \\ L^+ \triangleq \lambda(t : X \to S^+)(a : X)(b : X).L\, (t\, b)\, a, & c^+ \triangleq L, \\ \rho^+ \triangleq \lambda(t : S^+)(g : X \to X)(a : X).\rho\, (ta)\, g\, (ga), & r^+ \triangleq \rho\, (m\, S\, c\, z). \end{array}$$

A transition term can be written like this:

$$T \triangleq \lambda(p : P_X)(h : H_X).p\,(\lambda\, S\, K\, z\, L\, c\, \rho\, r.\, h\, S^+\, K^+\, z^+\, L^+\, c^+\, \rho^+\, r^+) : P_X \to P_X$$

Therefore, the following term computes modulus with expected behaviour:

$$\begin{aligned}\mathsf{Mod}_j \triangleq \lambda(m : \mathcal{N}_{j+1})(n : \mathcal{N}_{j+4})(X : \mathsf{rk}_j)(f : X \to X)(x : X).\\ n\, P_X\, T\, p_0\, (\lambda _______ r.r\, X\, f\, x).\end{aligned}$$

□

Remark. Note that the only difference between uniform and non-uniform representations is that, in non-uniform representations, we are allowed to pick different N_I and N_O for each function.

Corollary 4. *Unary functions in $\mathcal{E}^2$ are uniformly representable in* PTS-rk.

Proof. There is a binary function $\mathsf{ev} : \mathbb{N}^2 \to \mathbb{N}$ in $\mathcal{E}^3$ which is universal for unary functions in $\mathcal{E}^2$ [67, Section 5, p. 873, footnote 5]. In other words, for each $f : \mathbb{N} \to \mathbb{N}$ in $\mathcal{E}^2$, there is a code $\mathtt{f} \in \mathbb{N}$ such that

$$\forall n, \quad \mathsf{ev}(\mathtt{f}, n) = f(n).$$

And $\mathcal{E}^3$ is precisely elementary recursive functions, so by Theorem 10 there is a cost r and a term $E : \mathcal{N}_r \to \mathcal{N}_r \to \mathcal{N}_0$ such that

$$\forall mn, \quad E\,\overline{m}\,\overline{n} =_\beta \overline{\mathsf{ev}(m, n)}$$

Therefore, there exist $N_I \coloneqq \mathcal{N}_r$ and $N_O \coloneqq \mathcal{N}_0$ such that, for each unary $f \in \mathcal{E}^2$, there is a term $F \coloneqq E\,\overline{\mathtt{f}}$ such that

$$\forall n, \quad F\,\overline{n} =_\beta E\,\overline{\mathtt{f}}\,\overline{n} =_\beta \overline{\mathsf{ev}(\mathtt{f}, n)} = \overline{f(n)}.$$

□

Remark. While our system appears to be restricted to elementary recursive functions, turns out that there is another dependently typed system in which functions on natural numbers are *primitive* recursive [23]. Interestingly enough, part of their solution is in forbidding function types from appearing in the lowest universe, just like in PTS-rk.

There is also a boundary result which stems from information-theoretical considerations.

Theorem 11. *If a function $f : \mathbb{N}^k \to \mathbb{N}$ is representable in* PTS-rk*, it is asymptotically bounded above by an elementary recursive function of the size of input encoding.*

Proof. Since it is representable, there are $N_I : \mathsf{rk}_r$, $N_O : \mathsf{rk}_s$ and $F : N_I \to \ldots \to N_I \to N_O$ such that F represents f. Therefore,

$$\forall(n_1, \ldots, n_k), \quad F\,[n_1]^I \ldots [n_k]^I =_\beta [f(n_1, \ldots, n_k)]^O$$

Since terms are definitionally equal, they must have the same normal form which is then equal to $[f(n_1, \ldots, n_k)]^O$ as a word. Looking at the proof of Theorem 6, its length is then bounded above by $\mathcal{O}(c \uparrow \ldots \uparrow c \uparrow i(n_1, \ldots, n_k))$ where $i(n_1, \ldots, n_k)$ is length of $F\,[n_1]^I \ldots [n_k]^I$ as a word. Height of the tower of exponents depends not on the n_i-s, but on the d-bound which is equal to $r+1$ so this bound is ELEMENTARY.

By combinatorial considerations, length of $[x]^O$ is $\Omega(\log x)$ so

$$f(n_1, \ldots, n_k) = \mathcal{O}(\log(c \uparrow \ldots \uparrow c \uparrow i(n_1, \ldots, n_k))) = \\ = \mathcal{O}(c \uparrow \ldots \uparrow c \uparrow i(n_1, \ldots, n_k)),$$

thus bounded by an elementary recursive function of the size of input encoding. □

In particular case where elementary recursive encoding is used, function representable in PTS-rk is bounded by elementary recursive function.

Corollary 5. *Functions representable in* PTS-rk *with elementary recursive bound on input encoding size are precisely elementary recursive functions.*

Proof. Since functions computable under elementary recursive time bound are elementary recursive [27], this follows from Theorems 10, 11. □

Two questions remain open: a) whether there are non-elementary functions which are non-uniformly computable in PTS-rk because of non-elementary input encoding and b) whether there are any functions which are not in $\mathcal{E}^2$ and are uniformly computable in PTS-rk.

6 Set-theoretical model of PTS-rk

Now, studying computational properties of a calculus is great, but a logic is only useful when it can tell right from wrong. In this section, we are going to provide a nontrivial set-theoretical semantics for PTS-rk with respect to which it is sound, and where $[\![\Pi(X : \mathsf{rk}_j).X]\!]$ is going to be an empty set, as expected.

We are going to need an unusual assumption however:

Theorem 12. PTS-rk *has a sound model in ZF+GCH.* 🖒

Proof. Produced in discussion with GPT 6 Astra.

Take distinct codes c_κ for every cardinal κ below $\aleph_\omega$ and distinct additional codes s_j for $j \in \mathbb{N}$. Define

$$U_i = \{c_\kappa : \kappa \leq \aleph_i\} \cup \{s_j : j \in \mathbb{N}\}, \qquad \mathrm{El}(c_\kappa) = \kappa, \quad \mathrm{El}(s_j) = U_j.$$

That is,

$$\begin{aligned}
&U_0 = \{c_0, c_1, \ldots, c_\omega = c_{\aleph_0}, s_0, s_1, \ldots\},\\
&U_1 = \{c_0, c_1, \ldots, c_{\aleph_0}, c_{\aleph_1}, s_0, s_1, \ldots\},\\
&\ldots\\
&U_k = \{c_0, c_1, \ldots, c_{\aleph_0}, c_{\aleph_1}, \ldots, c_{\aleph_k}, s_0, s_1, \ldots\}\\
&\ldots
\end{aligned}$$

(Note that $x \in U_i \Rightarrow |\mathrm{El}(x)| \leq \aleph_i$ since all U_i are countable.)

Interpretation then goes as follows:

- Interpret rk_i by s_i. The sort axiom holds since $s_i \in U_0 = \mathrm{El}(s_0)$.
- If $a \in U_m$ and $b(x) \in U_n$ for every $x \in \mathrm{El}(a)$, the ordinary dependent function set has cardinality at most

$$\left| \prod_{x \in \mathrm{El}(a)} \mathrm{El}(b(x)) \right| \leq \aleph_n^{\aleph_m} \leq \aleph_{\max(m+1,n)}.$$

  (The last inequality follows from GCH since alephs have finite indices.)

  Thus, for all such a, b we can choose a cardinal $\kappa_{\Pi(x:a).b(x)}$ (whose code lies in $U_{\max(m+1,n)}$) along with a bijection

$$f : \left(\prod_{x \in \mathrm{El}(a)} \mathrm{El}(b(x)) \right) \simeq \kappa_{\Pi(x:a).b(x)}$$

  The eligible pairs (κ, f) form a set, so ordinary choice (which holds in ZF+GCH [64, 69]) suffices. Make each choice once for each (a, b), independently of a typing derivation or rank bound.
- Abstraction and application are modeled by f and f^{-1} respectively, as defined above.
- Contexts are interpreted by valuations.

It is easy to check that typing rules are respected by such interpretation, making it a sound model. □

A few corollaries:

- Indeed, the product interpreting $\Pi(Z : \mathsf{rk}_0).Z$ ranges over U_0, which contains c_0, whose decoding is $\varnothing$. That product is therefore empty.
- A system with rank cumulativity $t : \mathrm{rk}_j \Rightarrow t : \mathrm{rk}_{j+1}$ is sound with respect to this model; normalisation of bounded terms remains elementary as in Theorem 6.

Note that usage of GCH is unavoidable in this model to validate the rules. We could try building a model in ZFC using $\beth$-s instead of $\aleph$-s, but that would break an assumption that U_i are countable, which would falsify axioms $\mathsf{rk}_n : \mathsf{rk}_0$. That is not a huge problem however since ZF+GCH is interpretable in ZF [35, 36] (in contrast to ZFC + inaccessible cardinals which is not [45, 47, 37]).

7 PTS-rk as a proof system

To demonstrate ability of PTS-rk to embed nontrivial logical theories, we construct a fixed interpretation of Tarski–Mostowski–Robinson arithmetic R in PTS-rk: Church numerals and certified finite observations first interpret its relational form $\widetilde{\mathsf{R}}$; definable graphs yield R. We do so because interpretation of $\widetilde{\mathsf{R}}$ is much simpler than direct interpretation of R and already exposes all of the main ideas. Direct translation is available in supplementary formalised materials.

The following proof was obtained in discussion with GPT 6 Astra.

7.1 Classical arithmetic and the target calculus

Let $\widetilde{\mathsf{R}}$ be the relational presentation of Tarski–Mostowski–Robinson arithmetic in the first-order language with equality $\{0, S, =, \leq, A, M\}$, with ternary predicates A, M and numerals $\bar{n} = S^n 0$. Its axioms are the universal closures of the following schemes, indexed by external $m, n \in \mathbb{N}$:

$$A(\bar{m}, \bar{n}, x) \leftrightarrow \overline{m+n} = x, \tag{$\widetilde{R}1$}$$

$$M(\bar{m}, \bar{n}, x) \leftrightarrow \overline{mn} = x, \tag{$\widetilde{R}2$}$$

$$\bar{m} \neq \bar{n} \quad (m \neq n), \tag{$\widetilde{R}3$}$$

$$x \leq \bar{n} \leftrightarrow \bigvee_{i=0}^{n} x = \bar{i}, \tag{$\widetilde{R}4$}$$

$$x \leq \bar{n} \vee \bar{n} \leq x. \tag{$\widetilde{R}5$}$$

Here $m + n$ and mn are calculated externally: the language has no binary arithmetic function symbols. Section 7.7 relates this theory to the original, functional arithmetic R [72].

Let us start constructing the model inside PTS-rk and proving that axioms hold in our model. First, some preliminaries on interpretation of first-order logic formulas: put 🫳

$$\bot \triangleq (\Pi(Z : \mathsf{rk}_0).Z) : \mathsf{rk}_1, \qquad \top \triangleq \mathsf{rk}_0 \to \mathsf{rk}_0 : \mathsf{rk}_1, \qquad \mathsf{id} \triangleq \lambda(Z : \mathsf{rk}_0).Z : \top.$$

Use the fixed-answer proof types 🫳

$$\neg_c P \triangleq P \to \bot, \qquad P \wedge_c Q \triangleq (P \to Q \to \bot) \to \bot,$$
$$\bigvee_c(P_0, \ldots, P_k) \triangleq (P_0 \to \bot) \to \cdots \to (P_k \to \bot) \to \bot,$$
$$P \leftrightarrow_c Q \triangleq (P \to Q) \wedge_c (Q \to P).$$

A type P is *stable* when $\neg_c \neg_c P \to P$ is inhabited 🫳. Negative types, dependent functions with stable codomain, and these encoded conjunctions and disjunctions are stable. They support the classical logical rules on stable proof types; case elimination is into stable targets. In particular, $P \vee_c \neg_c P$ has the proof $\lambda(k : P \to \bot)(l : (P \to \bot) \to \bot).\, l\, k$. We never eliminate $\bot$ into an arbitrary positive type.

For a carrier D, a domain predicate Good, and fixed stable atomic interpretations, translate quantifiers by (with existential quantification abbreviating $\neg\forall\neg$) 🫳

$$(\forall x\, \varphi)^* \triangleq \Pi(x : D).\, \mathsf{Good}(x) \to \varphi^*(x),$$
$$(\exists x\, \varphi)^* \triangleq (\Pi(x : D).\, \mathsf{Good}(x) \to \varphi^*(x) \to \bot) \to \bot.$$

All translated formulas are stable. Once domain closure and atomic congruence are proved, the logical rules and equality substitution extend structurally to formulas on admitted arguments. All interpretations below use this one logical encoding.

For lifting propositions $P : \mathsf{rk}_a$ into arbitrary higher ranks $k \geq 2$, let

$$L_k(P) \triangleq (\Pi(T : \mathsf{rk}_{k-2}).T \to T) \to P : \mathsf{rk}_k.$$

Abstraction gives a map from P to $L_k(P)$; application on term $(\lambda\, T\, x.x)$ gives a map in the opposite direction.

7.2 Unary domain and equality

For this interpretation, use the argument order X, z, s for Church numerals, as in the Rocq development 🫳. Thus $D \triangleq \Pi(X : \mathsf{rk}_1).X \to (X \to X) \to X$ is the argument-permuted version of $\mathcal{N}_1$. Its two *constructors* are

$$\mathsf{z} \triangleq \lambda(X : \mathsf{rk}_1)(z : X)(s : X \to X).z, \qquad \mathsf{s}\, d \triangleq \lambda X\, z\, s.s(d\, X\, z\, s).$$

Use the same order for the auxiliary operations: $\mathsf{Add}\, d\, e \triangleq \lambda X\, z\, s.d\, X\, (e\, X\, z\, s)\, s$ and $\mathsf{Mul}\, d\, e \triangleq \lambda X\, z\, s.d\, X\, z\, (\lambda y : X.e\, X\, y\, s)$. Both have type $D \to D \to D$.

Even if the theory we interpret and the host type system are not strong enough to support induction on Church numerals, we still need to prove some properties by induction. For this, we introduce a special helper proposition as a domain predicate Good. It would state for a given $d : V$ that each inductive property Q is satisfied for d 🫳:

$$\mathsf{Good}(d) \triangleq \big(\Pi(Q : D \to \mathsf{rk}_{10}).\, Q(\mathsf{z}) \to (\Pi(u : D).\, Q(u) \to Q(\mathsf{s}u)) \to Q(d)\big) \\ : \mathsf{rk}_{12}.$$

Zero and successor preserve it by abstraction. For example,

$$\mathsf{good}_0 \triangleq \lambda Q\, q_0\, q_s.\, q_0, \qquad \mathsf{good}_s\, d\, g \triangleq \lambda Q\, q_0\, q_s.\, q_s\, d\, (g\, Q\, q_0\, q_s).$$

Define equality by negative tests 🫳:

$$E(d, e) \triangleq \Pi(P : D \to \mathsf{rk}_1).\, (\neg_c\, P(d)) \leftrightarrow_c (\neg_c\, P(e)) \quad : \mathsf{rk}_5.$$

It is reflexive, symmetric and transitive; every $F : D \to D$ respects it. Equality substitution for E itself follows from symmetry and transitivity. Define the arithmetic atoms by

$$A^*(d, e, f) \triangleq E(\mathsf{Add}\, d\, e, f), \qquad M^*(d, e, f) \triangleq E(\mathsf{Mul}\, d\, e, f).$$

They respect E in all arguments; axioms $\widetilde{R}1$ and $\widetilde{R}2$ hold definitionally.

7.3 Order from successor-closed unary tests

Definition 11 (🫳)**.** A *certified unary algebra* is the fixed telescope

$$\mathcal{U} = (X : \mathsf{rk}_1)\, (z : X)\, (s : X \to X)\, (\delta : X \to \mathsf{rk}_4) \\ (d_0 : \delta z)\, (d_s : \Pi(u : X).\, \delta u \to \delta(su)).$$

Basically, this telescope contains all the necessary information to "reinterpret" Church numerals into new carrier X with a domain predicate δ.

Writing $\mathcal{U} \to B$ abbreviates the successive dependent arrows for these binders; it does not introduce a record type. Put 🫳 $\mathrm{ev}_{\mathcal{U}}(d) = d\, X\, z\, s$ and $G(d) \triangleq (\mathcal{U} \to \delta(\mathrm{ev}_{\mathcal{U}}(d))) : \mathsf{rk}_6$. For $P : X \to \mathsf{rk}_1$, write

$$\mathsf{Up}_{\mathcal{U}}(P) \triangleq \Pi(u : X).\, \delta u \to \neg_c\, P(u) \to \neg_c\, P(su).$$

Meaning that, for "good" values in new domain X, a negation of P is upwards closed. Define 🫳

$$O(d,e) \triangleq \big(\mathcal{U} \to \Pi(P : X \to \mathsf{rk}_1).\ \mathsf{Up}_{\mathcal{U}}(P) \to \\ \neg_c P(\mathrm{ev}_{\mathcal{U}}(d)) \to \neg_c P(\mathrm{ev}_{\mathcal{U}}(e))\big) : \mathsf{rk}_6.$$

With such definition, preorder laws and equality compatibility are derivable.

Lemma 9 (🫳). *The following holds:*

- $G(\mathsf{z})$, $G(d) \to G(\mathsf{s}d)$ *and* $\mathsf{Good}(d) \to G(d)$ *are inhabited.*
- O *is reflexive and transitive.*
- $E(d,e) \to O(d,e)$ *and* $G(d) \to O(d, \mathsf{s}d)$ *are inhabited.*
- *Arguments of* O *respect* E*.*

Proof. Proof terms for most propositions are constructed in an obvious way. The last proposition holds because O is transitive and E is symmetric. □

Consequently, for each external $m \le n$, there is a closed proof of $O(\overline{m}, \overline{n})$, built by a finite succession of steps.

Remark. Actually, it is possible to prove inside PTS-rk that D satisfies axioms of an *ordered commutative unital semiring*. A generated formal proof is available in supplementary materials.

7.4 Finite separation

In this section we prove that $\widetilde{R}3$ also holds in our interpretation of $\widetilde{\mathsf{R}}$.

Lemma 10 (🫳). *For each* $n \in \mathbb{N}$*, there is a proof of* $O(\overline{n+1}, \overline{n}) \to \bot$.

Proof. By contradiction. Suppose $H : O(\overline{n+1}, \overline{n})$; put $c = n+1$, $q = c+1$, and remember already familiar type of selectors

$$F_q \triangleq \underbrace{\mathsf{rk}_1 \to \cdots \to \mathsf{rk}_1}_{q \text{ arguments}} \to \mathsf{rk}_1, \qquad e_i \triangleq \lambda Z_0 \ldots Z_{q-1}.Z_i.$$

We are going to use them as a "model" to interpret an assumed O and get a contradiction: set $X = F_q$, $z = e_0$, and use "saturated addition" as a successor:

$$s(u) \triangleq \lambda Z_0 \ldots Z_{q-1}.\ u\, Z_{\min(0+1,c)} \cdots Z_{\min(c+1,c)}.$$

Thus $s(e_i) =_\beta e_{\min(i+1,c)}$. Define the domain predicate as

$$\delta(u) \triangleq \big(\Pi(Q : F_q \to \mathsf{rk}_3).\ Q(e_0) \to \cdots \to Q(e_c) \to Q(u)\big) : \mathsf{rk}_4.$$

Selector projection gives d_0. To construct d_s, open the output $\delta(su)$ at Q and its branch proofs, and specialise the input certificate to $v \mapsto Q(sv)$. Its branches are the supplied proofs for the appropriate successor selectors.

Use the unary test

$$P(u) \triangleq u \underbrace{\top \cdots \top}_{c \text{ entries}} \bot.$$

To prove $\mathsf{Up}(P)$, eliminate $\delta(u)$ into the rank-three motive $v \mapsto \neg_c P(v) \to \neg_c P(sv)$. At e_c both tests are $\bot$ and the implication is identity. At e_i with $i < c$, there is a premise $\top \to \bot$.

Specialising $H : O(\overline{c}, \overline{n})$ to these data yields $(\bot \to \bot) \to (\top \to \bot)$. □

Lemma 11 (🤌)**.** *For each $m > n$, there is a proof of $O(\overline{m}, \overline{n}) \to \bot$.*

Proof. Suppose $O(\overline{m}, \overline{n})$ is inhabited; since $m > n$, $O(\overline{n+1}, \overline{m})$ is also inhabited. By transitivity of O it means that $O(\overline{n+1}, \overline{n})$ which we know is absurd. □

Thus, $\widetilde{R}3$ also holds: if $m \neq n$, then either $m > n$ or $m < n$ (these are external natural numbers). Assuming $E(\overline{m}, \overline{n})$, we obtain a paradoxical O and derive a contradiction by Lemma 11.

7.5 Bounded cases

Here, we prove a crucial lemma from which remaining axioms follow. Set 🤌

$$\mathsf{Case}_n(d) \triangleq \bigvee_c \big(E(d, \overline{0}), \dots, E(d, \overline{n}), O(\overline{n+1}, d)\big) : \mathsf{rk}_8.$$

Lemma 12 (🤌)**.** $\Pi(d : D).\ \mathsf{Good}(d) \to \mathsf{Case}_n(d)$ *is inhabited for external n.*

Proof. We are going to use the following positive invariant as a Q 🤌:

$$\mathsf{Inv}_n(d) \triangleq \Pi(Z : \mathsf{rk}_8).\ (G(d) \to \mathsf{Case}_n(d) \to Z) \to Z \quad : \mathsf{rk}_{10}.$$

At zero, supply $G(\mathsf{z})$ and the $E(\mathsf{z}, \overline{0})$ branch. At successor, open the target continuation and eliminate the child invariant at that same Z, obtaining $G(d)$ and $\mathsf{Case}_n(d)$. Obtain $G(\mathsf{s}d)$ independently. To construct the new negative case proof, all branch continuations return $\bot$. An $i < n$ branch becomes $i + 1$ by successor congruence. The $i = n$ branch gives $E(\mathsf{s}d, \overline{n+1})$, hence $O(\overline{n+1}, \mathsf{s}d)$. The large branch is composed with $O(d, \mathsf{s}d)$. This proves successor closure.

Instantiate $\mathsf{Good}(d)$ with Inv_n. Project its case component by choosing $Z = \mathsf{Case}_n(d)$ and continuation $\lambda g\, c.c$. No negative disjunction is eliminated into positive G-evidence. □

7.6 The relational interpretation

Theorem 13 (🫳). *The carrier D, admitted domain* Good*, closed operations* z, s*, and relations E, O, A^*, M^* interpret the classical relational theory $\widetilde{\mathsf{R}}$ in* PTS-rk*. Every translated axiom has an effectively constructible closed proof term. The interpretation is closed under classical first-order derivations: every $\widetilde{\mathsf{R}}$-derivation of a sentence effectively yields a closed* PTS-rk *proof term of its translation. True ground arithmetic atoms have proofs, and false ground atoms have internal refutations.*

The linked theorem gives original-PTS inhabitants for all axiom instances and all source derivations. The generic constructive witness search turns each such existence proof into a proof term together with its typing derivation 🫳.

Proof. Domain closure, atomic stability, and congruence have been established; hence source deductions translate to PTS proofs. We know that $\widetilde{R}1$-$\widetilde{R}3$ hold; let us check the remaining axioms.

For bounded order ($\widetilde{R}4$), apply Lemma 12. Under $O(d, \overline{n})$, a case with $O(\overline{n+1}, d)$ contradicts $\neg_c\, O(\overline{n+1}, \overline{n})$ by transitivity. Conversely, an $E(d, \overline{i})$ branch with $i \leq n$ gives $O(d, \overline{n})$ using $E \Rightarrow O$ and the ground comparison.

For comparability ($\widetilde{R}5$), the finite branches of Lemma 12 give $O(d, \overline{n})$ and the large branch gives $O(\overline{n}, d)$.

The ground proof/refutation claims follow from the arithmetic beta equations, reflexivity, ground order, and Lemma 11. □

Remark. Constructions from Sections 7.4 and 7.5 generalise to arbitrary Church-encoded inductive types providing constructor disjointness and injectivity, as well as finite case analysis for subtrees of fixed constructor trees. These results can be seen as internal inversion lemmas.

While this interpretation is full, it, however, is not faithful on arbitrary first-order sentences.

Remark (Failure of faithfulness 🫳). The interpretation of $\widetilde{\mathsf{R}}$ is not faithful.

Proof. The translation of $\rho \triangleq \forall x\, (x \leq x)$ is provable in PTS-rk, although $\widetilde{\mathsf{R}} \nvdash \rho$.

We already know that O is reflexive (Lemma 9). But why is reflexivity not derivable in $\widetilde{R}$? It follows from model-theoretic considerations: there is a set-theoretic countermodel.

Take $N^+ = \mathbb{N} \sqcup \{\omega\}$, where ω is one new element. Interpret 0 and successor as usual on $\mathbb{N}$, put $S\omega = \omega$, and use actual equality. For $x, y \in \mathbb{N}$, let $A(x, y, z)$ hold exactly when $z = x + y$, and let $M(x, y, z)$ hold exactly when $z = xy$. When at least one of x, y is ω, let each predicate hold exactly when $z = 0$. Define

$$x \leq^+ y \quad \Longleftrightarrow \quad \big(x, y \in \mathbb{N} \text{ and } x \leq_{\mathbb{N}} y\big) \text{ or } \big(x \in \mathbb{N} \text{ and } y = \omega\big).$$

The ground graph schemes hold for every third argument, and distinct numerals are unequal. For every standard n, the downset of n is exactly $\{0, \dots, n\}$, proving both directions of $(\widetilde{R}4)$. Every standard number is comparable with n, and $n \leq^+ \omega$, proving $(\widetilde{R}5)$. Hence this is a model of $\widetilde{\mathsf{R}}$.

However, $\omega \not\leq^+ \omega$, so ρ is false. □

7.7 Relation to the original presentation of R

The original Tarski–Mostowski–Robinson arithmetic R [72] uses $\{0, S, +, \cdot, \leq\}$ and the following schemes, again with universal closure:

$$\bar{m} + \bar{n} = \overline{m+n}, \tag{R1}$$

$$\bar{m} \cdot \bar{n} = \overline{mn}, \tag{R2}$$

$$\bar{m} \neq \bar{n} \quad (m \neq n), \tag{R3}$$

$$x \leq \bar{n} \longrightarrow \bigvee_{i=0}^{n} x = \bar{i}, \tag{R4}$$

$$x \leq \bar{n} \vee \bar{n} \leq x. \tag{R5}$$

Requiring A, M in $\widetilde{\mathsf{R}}$ to be total, single-valued graphs makes them interchangeable with the function symbols $+, \cdot$. This functional strengthening is definitionally equivalent to R; even without that strengthening, the two theories have the same interpretability degree.

Proposition 13. *The classical theories* $\widetilde{\mathsf{R}}$ *and* R *are mutually interpretable* [2].

Proof. In R, interpret $A(x, y, z)$ by $x + y = z$ and $M(x, y, z)$ by $xy = z$. The ground arithmetic equations give the two graph schemes. For $i < j$, an assumption $\bar{j} \leq \bar{i}$ contradicts (R4) and numeral separation; (R5) therefore yields $\bar{i} \leq \bar{j}$. The case $i = j$ follows from (R5) directly. Thus R proves all true ground comparisons, and equality substitution supplies the reverse implication in $(\widetilde{R}4)$. The remaining relational schemes are immediate.

For the other direction define, in $\widetilde{\mathsf{R}}$,

$$U_A(x, y, z) \triangleq A(x, y, z) \wedge \forall w \big(A(x, y, w) \to w = z\big),$$
$$F_A(x, y, z) \triangleq U_A(x, y, z) \vee \big(z = 0 \wedge \neg \exists v\, U_A(x, y, v)\big).$$

For fixed x, y, either a unique A-output exists or it does not. In the first case F_A selects that output; in the second it selects zero. Hence F_A is provably total and single-valued. At standard inputs, $(\widetilde{R}1)$ forces its value to be $\overline{m+n}$. Define F_M identically with M in place of A; its standard value is $\overline{mn}$. These graphs interpret $+, \cdot$; keep $0, S, =, \leq$ unchanged. All five schemes of R follow.

[2] Mutual interpretability does not imply bi-interpretability. In particular, the totalised graphs need not coincide with the original relations A, M on arbitrary arguments.

Composing this graph interpretation with Theorem 13 gives the claimed interpretation in PTS-rk. The composition uses the encoded classical quantifiers to express values and uniqueness; it does not assert Good-closure of the raw Church operators $\mathsf{Add}, \mathsf{Mul}$. □

Corollary 6. PTS-rk *interprets* R *by definable arithmetic graphs.*

8 Conclusion

To sum it all up, what started as a funny idea (what if there actually was a sound pure type system with $\mathsf{type} : \mathsf{type}$) led us to stumble on a truly unique dependently typed pure type system, PTS-rk.

First of all, its normalisation proof is suspiciously simple and reminds of classic proofs of normalisation for STLC where, instead of an established technique by reducibility candidates, one patiently tracks complexities of arrow types and makes sure that they become simpler after sequences of reductions (Section 3). To make sure that this approach works for our system, we formalised our proof in Rocq proof assistant (Section 3.1) and in doing so prepared the groundwork for future verified interactions with PTS-rk.

After noticing that normalisation proof yields a primitive recursive normalisation procedure, we found tighter bounds and found out that it has the same complexity as normalisation of STLC, which was previously unheard of for a dependently typed system (Section 4). In contrast with existing pure type systems, our system is able to represent elementary recursive functions (both in the sense that it is stronger than STLC and weaker than more classical dependently typed systems) and seems intimately connected to it (Section 5). To check if constructions inside PTS-rk even make sense, we constructed a set-theoretical model for it which assumes a Generalised Continuum Hypothesis in order to strictify the hierarchy of $\aleph$-s which act as interpretations of our function types (Section 6). Finally, we provide a lower bound on proof-theoretical strength of PTS-rk by providing a full interpretation of a relational version of Tarski–Mostowski–Robinson arithmetic R inside it (Section 7); we did it partly because we suspect our system to be able to prove its own consistency in the sense Pakhomov finitistic set theory does [55], therefore interpretation of R is the natural first step.

References

[1] Andreas Abel, Joakim Öhman, and Andrea Vezzosi. "Decidability of Conversion for Type Theory in Type Theory". In: *Proceedings of the ACM on Programming Languages* 2.POPL (2018), 23:1–23:29. DOI: 10.1145/3158111.

[2] Peter Aczel. “On Relating Type Theories and Set Theories”. In: *Types for Proofs and Programs (TYPES 1998)*. Vol. 1657. Lecture Notes in Computer Science. Springer, 1999, pp. 1–18. DOI: 10.1007/3-540-48167-2_1.

[3] Robin Adams. “Pure Type Systems with Judgemental Equality”. In: *Journal of Functional Programming* 16.2 (2006), pp. 219–246. DOI: 10.1017/S0956796805005770.

[4] Thorsten Altenkirch. “A Formalization of the Strong Normalization Proof for System F in LEGO”. In: *Typed Lambda Calculi and Applications (TLCA 1993)*. Vol. 664. Lecture Notes in Computer Science. Springer, 1993, pp. 13–28. DOI: 10.1007/BFb0037095.

[5] Thorsten Altenkirch. “Constructions, Inductive Types and Strong Normalization”. PhD thesis. University of Edinburgh, Nov. 1993.

[6] Thorsten Altenkirch, James Chapman, and Tarmo Uustalu. “Monads Need Not Be Endofunctors”. In: *Logical Methods in Computer Science* 11.1 (2015). DOI: 10.2168/LMCS-11(1:3)2015.

[7] Thorsten Altenkirch and Bernhard Reus. “Monadic Presentations of Lambda Terms Using Generalized Inductive Types”. In: *Computer Science Logic (CSL 1999)*. Vol. 1683. Lecture Notes in Computer Science. Springer, 1999, pp. 453–468. DOI: 10.1007/3-540-48168-0_32.

[8] Anthropic. *Claude Fable 5*. Large language model. Version Mythos-class. 2026. URL: https://claude.ai/ (visited on 10/13/2026).

[9] Henk P. Barendregt. “Introduction to Generalized Type Systems”. In: *Journal of Functional Programming*. Vol. 1. 2. Cambridge University Press, 1991, pp. 125–154. DOI: 10.1017/S0956796800020025.

[10] Henk P. Barendregt. “Lambda Calculi with Types”. In: *Handbook of Logic in Computer Science*. Ed. by Samson Abramsky, Dov M. Gabbay, and Thomas S. E. Maibaum. Vol. 2. Oxford University Press, 1992, pp. 117–309.

[11] Henk P. Barendregt. *The Lambda Calculus: Its Syntax and Semantics*. Revised. Vol. 103. Studies in Logic and the Foundations of Mathematics. North-Holland, 1984.

[12] Bruno Barras. “Auto-validation d’un système de preuves avec familles inductives”. PhD thesis. Université Paris 7, 1999.

[13] Bruno Barras and Benjamin Werner. *Coq in Coq*. Manuscript. INRIA Rocquencourt, 1997.

[14] Arnold Beckmann. “Exact Bounds for Lengths of Reductions in Typed λ-Calculus”. In: *Journal of Symbolic Logic* 66.3 (2001), pp. 1277–1285. DOI: 10.2307/2695106.

[15] Françoise Bellegarde and James Hook. “Substitution: A Formal Methods Case Study Using Monads and Transformations”. In: *Science of Computer Programming* 23.2–3 (1994), pp. 287–311. DOI: 10.1016/0167-6423(94)00022-0.

[16] L. S. van Benthem Jutting. “Typing in Pure Type Systems”. In: *Information and Computation* 105.1 (1993), pp. 30–41. DOI: 10.1006/inco.1993.1038.

[17] Stefano Berardi. *Towards a Mathematical Analysis of the Coquand–Huet Calculus of Constructions and the Other Systems in Barendregt's Cube*. Technical report. Department of Computer Science, Carnegie Mellon University, and Dipartimento di Matematica, Università di Torino, 1988.

[18] Yves Bertot and Pierre Castéran. *Interactive Theorem Proving and Program Development: Coq'Art: The Calculus of Inductive Constructions*. Texts in Theoretical Computer Science. Springer, 2004. DOI: 10.1007/978-3-662-07964-5.

[19] Richard S. Bird and Ross Paterson. "De Bruijn Notation as a Nested Datatype". In: *Journal of Functional Programming* 9.1 (1999), pp. 77–91. DOI: 10.1017/S0956796899003366.

[20] Corrado Böhm and Alessandro Berarducci. "Automatic Synthesis of Typed Λ-Programs on Term Algebras". In: *Theoretical Computer Science* 39 (1985), pp. 135–154. DOI: 10.1016/0304-3975(85)90135-5.

[21] Nicolaas G. de Bruijn. "Lambda Calculus Notation with Nameless Dummies, a Tool for Automatic Formula Manipulation, with Application to the Church–Rosser Theorem". In: *Indagationes Mathematicae* 34.5 (1972), pp. 381–392. DOI: 10.1016/1385-7258(72)90034-0.

[22] Nicolaas G. de Bruijn. "The Mathematical Language AUTOMATH, Its Usage, and Some of Its Extensions". In: *Symposium on Automatic Demonstration*. Vol. 125. Lecture Notes in Mathematics. Springer, 1970, pp. 29–61. DOI: 10.1007/BFb0060623.

[23] Ulrik Torben Buchholtz and Johannes Schipp von Branitz. "Primitive Recursive Dependent Type Theory". In: *Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science*. LICS '24. Tallinn, Estonia: Association for Computing Machinery, 2024. ISBN: 9798400706608. DOI: 10.1145/3661814.3662136.

[24] Alonzo Church. "A Formulation of the Simple Theory of Types". In: *Journal of Symbolic Logic* 5.2 (1940), pp. 56–68. DOI: 10.2307/2266170.

[25] Alonzo Church. *The Calculi of Lambda-Conversion*. Vol. 6. Annals of Mathematics Studies. Princeton University Press, 1941.

[26] Alonzo Church and J. Barkley Rosser. "Some Properties of Conversion". In: *Transactions of the American Mathematical Society* 39.3 (1936), pp. 472–482. DOI: 10.2307/1989762.

[27] Stephen A. Cook and Bruce Kapron. *A Survey of Classes of Primitive Recursive Functions*. ECCC Report TR17-001. Transcribed by Bruce Kapron from Stephen A. Cook's mimeographed course notes for Math 290, Section 14, University of California, Berkeley, January 1967. Electronic Colloquium on Computational Complexity, Jan. 2017. URL: https://eccc.weizmann.ac.il/report/2017/001/.

[28] Thierry Coquand and Jean Gallier. "A Proof of Strong Normalization for the Theory of Constructions Using a Kripke-like Interpretation". In: *Proceedings of the First Workshop on Logical Frameworks (Antibes)*. Ed. by Gérard Huet and Gordon Plotkin. 1990, pp. 479–497.

[29] Thierry Coquand and Gérard Huet. "The Calculus of Constructions". In: *Information and Computation* 76.2–3 (1988), pp. 95–120. DOI: 10.1016/0890-5401(88)90005-3.

[30] Thiago Felicissimo. "Generic Bidirectional Typing for Dependent Type Theories". In: *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer, 2024, pp. 89–107. DOI: 10.1007/978-3-031-57262-3_6.

[31] Steven Fortune, Daniel Leivant, and Michael O'Donnell. "The Expressiveness of Simple and Second-Order Type Structures". In: *Journal of the ACM* 30.1 (1983), pp. 151–185. DOI: 10.1145/322358.322370.

[32] Herman Geuvers and Mark-Jan Nederhof. "A Modular Proof of Strong Normalization for the Calculus of Constructions". In: *Journal of Functional Programming* 1.2 (1991), pp. 155–189. DOI: 10.1017/S0956796800020037.

[33] Jean-Yves Girard. "Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur". Thèse d'État. Université Paris VII, 1972.

[34] Jean-Yves Girard, Yves Lafont, and Paul Taylor. *Proofs and Types*. Vol. 7. Cambridge Tracts in Theoretical Computer Science. Cambridge University Press, 1989.

[35] Kurt Gödel. "The Consistency of the Axiom of Choice and of the Generalized Continuum-Hypothesis". In: *Proceedings of the National Academy of Sciences of the United States of America* 24.12 (1938), pp. 556–557. DOI: 10.1073/pnas.24.12.556.

[36] Kurt Gödel. *The Consistency of the Axiom of Choice and of the Generalized Continuum-Hypothesis with the Axioms of Set Theory*. Vol. 3. Annals of Mathematics Studies. Princeton University Press, 1940.

[37] Kurt Gödel. "Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I". In: *Monatshefte für Mathematik und Physik* 38 (1931), pp. 173–198. DOI: 10.1007/BF01700692.

[38] Robert Harper and Robert Pollack. "Type Checking with Universes". In: *Theoretical Computer Science* 89.1 (1991), pp. 107–136. DOI: 10.1016/0304-3975(90)90108-T.

[39] Felix Hausdorff. "Grundzüge einer Theorie der geordneten Mengen". In: *Mathematische Annalen* 65 (1908), pp. 435–505. DOI: 10.1007/BF01451165.

[40] Antonius J. C. Hurkens. "A Simplification of Girard's Paradox". In: *Typed Lambda Calculi and Applications (TLCA 1995)*. Vol. 902. Lecture Notes in Computer Science. Springer, 1995, pp. 266–278. DOI: 10.1007/BFb0014058.

[41] Thomas Jech. *Set Theory*. Third Millennium. Springer Monographs in Mathematics. Springer, 2003. DOI: 10.1007/3-540-44761-X.

[42] Felix Joachimski and Ralph Matthes. "Short Proofs of Normalization for the Simply-Typed λ-Calculus, Permutative Conversions and Gödel's T". In: *Archive for Mathematical Logic* 42.1 (2003), pp. 59–87. DOI: 10.1007/s00153-002-0156-9.

[43] Felix Joachimski and Ralph Matthes. "Short proofs of normalization for the simply-typed λ-calculus, permutative conversions and Gödel's T". In: *Archive for Mathematical Logic* 42.1 (2003), pp. 59–87. DOI: 10.1007/

s00153-002-0156-9. URL: https://doi.org/10.1007/s00153-002-0156-9.

[44] László Kalmár. "Egyszerű példa eldönthetetlen aritmetikai problémára". In: *Matematikai és Fizikai Lapok* 50 (1943). In Hungarian; introduces the elementary functions, pp. 1–23.

[45] Akihiro Kanamori. *The Higher Infinite: Large Cardinals in Set Theory from Their Beginnings*. Second. Springer Monographs in Mathematics. Springer, 2003. DOI: 10.1007/978-3-540-88867-3.

[46] Stephen C. Kleene. *Introduction to Metamathematics*. Amsterdam: North-Holland, 1952.

[47] Kenneth Kunen. *Set Theory: An Introduction to Independence Proofs*. Vol. 102. Studies in Logic and the Foundations of Mathematics. North-Holland, 1980.

[48] Gyesik Lee and Benjamin Werner. "Proof-Irrelevant Model of CC with Predicative Induction and Judgmental Equality". In: *Logical Methods in Computer Science* 7.4 (2011). DOI: 10.2168/LMCS-7(4:5)2011.

[49] Zhaohui Luo. *Computation and Reasoning: A Type Theory for Computer Science*. Vol. 11. International Series of Monographs on Computer Science. Oxford University Press, 1994.

[50] Per Martin-Löf. "A Theory of Types". Preprint 71-3, Department of Mathematics, University of Stockholm. 1971.

[51] Per Martin-Löf. "An Intuitionistic Theory of Types: Predicative Part". In: *Logic Colloquium '73*. Ed. by H. E. Rose and J. C. Shepherdson. Vol. 80. Studies in Logic and the Foundations of Mathematics. North-Holland, 1975, pp. 73–118. DOI: 10.1016/S0049-237X(08)71945-1.

[52] Per Martin-Löf. *Intuitionistic Type Theory*. Studies in Proof Theory. Notes by Giovanni Sambin of a series of lectures given in Padua, June 1980. Naples: Bibliopolis, 1984.

[53] Lê Thành Dũng Nguyên. "Simply typed convertibility is TOWER-complete even for safe λ-terms". In: *Logical Methods in Computer Science* 20.3 (Sept. 5, 2024). DOI: 10.46298/lmcs-20(3:21)2024. arXiv: 2305.12601. URL: https://lmcs.episciences.org/14215.

[54] OpenAI. *GPT-6 Astra*. Version Astra. Large language and embodied AI model. 2026. URL: https://openai.com.

[55] Fedor Pakhomov. "A weak set theory that proves its own consistency". In: *arXiv preprint arXiv:1907.00877* (2019). arXiv: 1907.00877 [math.LO].

[56] Rózsa Péter. *Rekursive Funktionen*. Budapest: Akadémiai Kiadó, 1951.

[57] Mihai Prunescu, Lorenzo Sauras-Altuzarra, and Joseph M. Shunia. *A Minimal Substitution Basis for the Kalmár Elementary Functions*. 2026. arXiv: 2505.23787 [math.LO]. URL: https://arxiv.org/abs/2505.23787.

[58] Femke van Raamsdonk and Paula Severi. *On Normalisation*. Tech. rep. CS-R9545. Amsterdam: CWI, 1995.

[59] Femke van Raamsdonk et al. "Perpetual Reductions in λ-Calculus". In: *Information and Computation* 149.2 (1999), pp. 173–225. DOI: 10.1006/inco.1998.2750.

[60] H. E. Rose. *Subrecursion: Functions and Hierarchies*. Vol. 9. Oxford Logic Guides. Oxford: Clarendon Press, 1984.

[61] J. Barkley Rosser. "Highlights of the History of the Lambda-Calculus". In: *Annals of the History of Computing* 6.4 (1984), pp. 337–349. DOI: 10.1109/MAHC.1984.10040.

[62] Helmut Schwichtenberg. "An Upper Bound for Reduction Sequences in the Typed λ-Calculus". In: *Archive for Mathematical Logic* 30.5–6 (1991), pp. 405–408. DOI: 10.1007/BF01621476.

[63] Helmut Schwichtenberg. "Definierbare Funktionen im λ-Kalkül mit Typen". In: *Archiv für mathematische Logik und Grundlagenforschung* 17.3–4 (1976), pp. 113–114. DOI: 10.1007/BF02276799.

[64] Wacław Sierpiński. "L'hypothèse généralisée du continu et l'axiome du choix". In: *Fundamenta Mathematicae* 34 (1947), pp. 1–5. DOI: 10.4064/fm-34-1-1-5.

[65] Vincent Siles and Hugo Herbelin. "Pure Type System Conversion is Always Typable". In: *Journal of Functional Programming* 22.2 (2012), pp. 153–180. DOI: 10.1017/S0956796812000044.

[66] Thoralf Skolem. "The foundations of elementary arithmetic established by means of the recursive mode of thought, without the use of apparent variables ranging over infinite domains". In: *From Frege to Gödel: A Source Book in Mathematical Logic, 1879–1931*. Ed. and trans. by Jean van Heijenoort. Original work published in German in 1923 as *Begründung der elementaren Arithmetik durch die rekurrierende Denkweise ohne Anwendung scheinbarer Veränderlichen, die unendliche Bereiche durchlaufen*. Cambridge, Massachusetts: Harvard University Press, 1967, pp. 302–333.

[67] Dimiter Skordev. "On the Subrecursive Computability of Several Famous Constants". In: *Journal of Universal Computer Science* 14.6 (2008), pp. 861–875. DOI: 10.3217/jucs-014-06-0861. URL: https://www.jucs.org/jucs_14_6/on_the_subrecursive_computability/jucs_14_06_0861_0875_skordev.pdf.

[68] Matthieu Sozeau and Nicolas Tabareau. "Universe Polymorphism in Coq". In: *Interactive Theorem Proving (ITP 2014)*. Vol. 8558. Lecture Notes in Computer Science. Springer, 2014, pp. 499–514. DOI: 10.1007/978-3-319-08970-6_32.

[69] Ernst Specker. "Verallgemeinerte Kontinuumshypothese und Auswahlaxiom". In: *Archiv der Mathematik* 5 (1954), pp. 332–337. DOI: 10.1007/BF01898380.

[70] Richard Statman. "The Typed λ-Calculus is Not Elementary Recursive". In: *Theoretical Computer Science* 9.1 (1979), pp. 73–81. DOI: 10.1016/0304-3975(79)90007-0.

[71] William W. Tait. "Intensional Interpretations of Functionals of Finite Type I". In: *Journal of Symbolic Logic* 32.2 (1967), pp. 198–212. DOI: 10.2307/2271658.

[72] Alfred Tarski, Andrzej Mostowski, and Raphael M. Robinson. *Undecidable Theories*. Studies in Logic and the Foundations of Mathematics. Amsterdam: North-Holland Publishing Company, 1953, pp. ix+98.

[73] Jan Terlouw. “Een nadere bewijstheoretische analyse van GSTT’s”. Manuscript, Faculty of Mathematics and Computer Science, University of Nijmegen. 1989.

[74] The Rocq Development Team. *The Rocq Prover Reference Manual*. Version 9.0 (formerly the Coq Proof Assistant). `https://rocq-prover.org/doc`. Inria. 2025. DOI: `10.5281/zenodo.1003420`.

[75] Amin Timany and Matthieu Sozeau. *Consistency of the Predicative Calculus of Cumulative Inductive Constructions (pCuIC)*. Research report RR-9105. KU Leuven and Inria, 2017.

[76] Amin Timany and Matthieu Sozeau. “Cumulative Inductive Types in Coq”. In: *3rd International Conference on Formal Structures for Computation and Deduction (FSCD 2018)*. Vol. 108. LIPIcs. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018, 29:1–29:16. DOI: `10.4230/LIPIcs.FSCD.2018.29`.

[77] Benjamin Werner. “Sets in Types, Types in Sets”. In: *Theoretical Aspects of Computer Software (TACS 1997)*. Vol. 1281. Lecture Notes in Computer Science. Springer, 1997, pp. 530–546. DOI: `10.1007/BFb0014566`.